

HADOOP & SPARK — COMPLETE COURSE NOTES

Lectures 1 – 5 · Spark Lecture 1

LECTURE 1 — INTRODUCTION TO BIG DATA & HADOOP

1. Why Did “Big Data” Even Come?

Before Hadoop, companies already had databases, servers, storage systems, SQL, and applications. So what suddenly became a problem?

The answer: Data started growing faster than traditional systems could handle.

2. What Exactly is “Big Data”?

Big Data does NOT simply mean “huge amount of data” — that is only one part.

Big Data means: data that becomes difficult to store, process, manage, analyze, or move using traditional systems.

So even 100 GB can become Big Data for one company if their systems cannot handle it properly. For Google, even Petabytes may be normal.

Big Data is relative.

3. Big Data is a “Problem Category”

Big Data is basically a collection of problems related to data — not just “large files.”

4. Traditional Systems vs Big Data Systems

Traditional Systems

Worked well when data was smaller, users were fewer, processing was manageable, and data types were mostly structured.

Examples: Banking software, College database, Inventory management.

Usually: one server, one database, vertical scaling.

5. Then Companies Like Google Faced New Problems

Google had billions of searches, web crawling, images, videos, user logs, emails, ads, and maps.

Traditional databases started failing because: storage became insufficient, processing became too slow, single servers crashed, scaling became expensive, and data arrived too fast.

This created the Big Data revolution.

6. The Famous 5 V's of Big Data

V1 — Volume

Means: Huge amount of data. Example: YouTube videos, Netflix streaming logs, Instagram images, banking transactions.

Traditional systems struggle because storage fills quickly, query becomes slow, and backup becomes difficult.

V2 — Velocity

Means: Speed at which data is generated or processed. Example: stock market updates, live GPS tracking, social media feeds, IoT sensors.

Data comes continuously every second. Problem: traditional systems cannot process fast enough.

V3 — Variety

Means: Different types/formats of data. Traditional databases mostly handled tables, rows, columns.

But companies started generating: videos, audio, images, JSON, logs, emails, PDFs, sensor data.

V4 — Veracity

Means: Data quality and trustworthiness. Real-world data is messy — missing values, duplicate records, fake

reviews, wrong sensor readings.

Problem: Bad data → bad decisions.

V5 — Value

Means: Whether the data is actually useful. Companies may store Petabytes of data, but is it useful? Can it improve business? Can it increase profit?

If not: Storage is meaningless.

Important Clarification

You said: “The 5Vs are problems.” Correct — more accurately, the 5Vs describe the challenges that make data become Big Data.

7. Big Data Engineer — Real Industry Understanding

A Big Data Engineer is NOT defined only by Petabytes. Instead, they solve large-scale data problems relative to the organization.

Example: Small companies — 100 GB may feel huge, their systems may struggle. Google — 100 GB is tiny. Still, both can use Big Data technologies.

8. So What Does a Big Data Engineer Actually Do?

- Storage systems
- Distributed systems
- Data pipelines
- Data movement
- Data processing
- Scalability
- Reliability
- Fault tolerance

9. Different Layers in the Data Ecosystem

Layer	Description / Examples
1. Data Generation	Apps, Websites, Sensors, Mobile devices, Logs, APIs, Databases
2. Data Ingestion	Moving data into systems — Kafka, Flume, Sqoop
3. Data Storage	HDFS, S3, Data Lake, Databases
4. Data Processing	MapReduce, Spark, Hive
5. Data Pipeline Engineering	ETL — Extract, Clean, Transform, Load
6. Scheduling & Orchestration	Airflow, Oozie
7. Analytics & BI	Dashboards, Reports, Insights — Power BI, Tableau
8. Machine Learning & AI	Recommendation systems, Fraud detection, Deep learning

10. Hadoop History — Deep Understanding

Before Hadoop, Google faced enormous scaling issues and traditional systems failed. So Google invented:

- A. Google File System (GFS)
- B. Google MapReduce

Around 2003–2004.

What Was Google File System (GFS)?

Google needed massive storage, fault tolerance, and distributed storage — so they created a distributed file system.

What is Distributed?

Distributed means work is spread across multiple machines. Instead of one giant computer, Google used

thousands of cheaper computers.

Instead of: 1 machine storing 1000 TB

Use: 1000 machines storing 1 TB each

Advantages: Cheaper, Faster, Easier scaling, Fault tolerant.

What is Fault Tolerance?

Suppose 1000 computers are working — some computers WILL fail. Big systems assume machines will crash, hard disks will fail, network issues will happen.

So systems must continue working despite failures. This is called Fault Tolerance.

Google MapReduce

Storage alone was not enough — Google also needed massive processing, so they created MapReduce, a distributed processing model.

Simple Understanding of MapReduce

Suppose you want to count words in 10 TB of files. Instead of 1 computer processing everything, MapReduce splits work, sends pieces to many machines, processes in parallel, and combines results.

This dramatically increases speed.

Then Came Hadoop

Doug Cutting and Mike Cafarella were working on Nutch (a search engine project). They studied Google's papers on GFS and MapReduce, then recreated similar ideas in open-source form. This became Hadoop.

Why the Name “Hadoop”?

Doug Cutting named it after his child's toy elephant. So Hadoop has no technical meaning.

Hadoop Core Components

Component	Inspired By
HDFS (Hadoop Distributed File System)	Google File System
MapReduce	Google MapReduce

11. What is Apache?

Apache Software Foundation (ASF) is a nonprofit organization. It manages open-source projects, communities, licensing, and governance.

12. What is Open Source?

Open source means source code is publicly available. People can view, modify, use, and contribute improvements to the code.

What is Source Code?

The actual program code written by developers — e.g. Python files, Java files, C++ files.

13. Closed Source vs Open Source

Closed Source	Open Source
Examples: Windows, Photoshop	Examples: Hadoop, Linux, Spark
Code is private, only company controls it	Code is public, community contributes

14. Then How Do Open Source Companies Make Money?

People often think: "If software is free, how do companies earn?"

Answer: They earn from Support, Enterprise tools, Security, Monitoring, Cloud hosting, Consulting, Maintenance — NOT from basic software alone.

What is “Support”?

Support means professional help for companies using software. Example: a bank uses Hadoop. Problems happen — cluster crash, security issue, data corruption, performance tuning — the company needs experts.

Support companies provide: Installation, Maintenance, Troubleshooting, Upgrades, Security patches, Monitoring — this is called enterprise support.

What is Enterprise?

Enterprise means large organizations/businesses — e.g. Banks, Amazon, Telecom companies, Hospitals.

Enterprise systems require Stability, Security, Reliability, Professional support.

Why Enterprises Don't Directly Use Raw Open Source

Raw Apache Hadoop may require manual configuration, expert setup, complex maintenance. Enterprises prefer easier tools, support teams, monitoring dashboards, security integrations. So companies build enterprise editions.

What is Enterprise Edition?

A commercial, business-ready version of open-source software. It usually includes GUI dashboards, monitoring tools, security tools, technical support, easier deployment, performance optimization.

Example

Apache Hadoop is open source. But companies created enterprise distributions.

15. Cloudera & Hortonworks

Two famous Hadoop companies: Cloudera and Hortonworks. They took Apache Hadoop, added enterprise features, provided support, and sold services to companies.

Important: They did NOT “own” Hadoop. Apache owned the open-source project — these companies built business products around it.

What is a Hadoop Distribution?

Like Linux distributions (kernel + tools + UI + support). Similarly, a Hadoop distribution = Hadoop + additional tools + security + management UI + monitoring + enterprise support, packaged together.

What is “Open Core”?

Open Core means basic software is open source, advanced enterprise features are paid.

Free: Basic Hadoop

Paid: Advanced monitoring, Enterprise security, Premium support

16. AWS EMR

Entity: Amazon Web Services. EMR = Elastic MapReduce.

AWS provides managed Hadoop/Spark clusters in the cloud. Instead of buying servers physically, companies rent clusters from the cloud.

Why Cloud Became Popular

Maintaining Hadoop clusters physically is difficult because of hardware, cooling, networking, failures, scaling. Cloud simplified this.

17. Evolution of the Big Data Ecosystem

Initially: Hadoop + MapReduce. Then: Spark, Kafka, Hive, Airflow, Cloud platforms became dominant.

Today, Spark is often preferred over MapReduce because it's faster, uses in-memory processing, and has easier APIs.

Important Final Big Picture

Big Data is NOT just: Hadoop, Huge files.

It is an entire ecosystem for solving: Storage problems, Processing problems, Scalability problems, Distributed computing problems, Real-time data problems.

18. Final Mental Model

Huge Data Problems



Traditional Systems Fail



Distributed Systems Needed
↓
Google Invented GFS + MapReduce
↓
Doug Cutting Created Hadoop
↓
Apache Open Source Ecosystem
↓
Companies Built Enterprise Solutions
↓
Modern Big Data Ecosystem Emerged

19. Super Important Revision Summary

Term	Meaning
Big Data	Data difficult to store/process/manage traditionally
5V's	Volume, Velocity, Variety, Veracity, Value
Hadoop	Open-source distributed storage + processing framework
HDFS	Distributed storage
MapReduce	Distributed processing
Apache Software Foundation	Nonprofit managing open-source projects
Open Source	Publicly available code
Enterprise Edition	Business-ready supported version
Cloudera / Hortonworks	Companies selling Hadoop support/services
Support	Professional maintenance/help for enterprises
AWS EMR	Cloud-managed Hadoop ecosystem

LECTURE 2 — UNBOXING HADOOP FRAMEWORK / ECOSYSTEM ARCHITECTURE

Chapter 1: Hadoop = Storage + Processing

Hadoop works like a human body:

- HDFS = Stomach (Storage) — stores data
- MapReduce = Brain (Processing) — processes data

Without storage: no data exists. Without processing: data is useless.

Original Hadoop architecture (Hadoop Core): HDFS + MapReduce.

Why Only These Two?

Google's papers solved two major problems:

- Problem 1: Where to store Petabytes? → Google File System (GFS) → converted to HDFS
- Problem 2: How to process Petabytes? → Google MapReduce → converted to MapReduce

Chapter 2: Why Linux?

Most Hadoop clusters run on Linux because:

- Open Source — No license cost
- Stable — Servers run for years
- Secure — Better permission control
- Lightweight — Consumes less memory
- Highly Customizable — Can be modified

Chapter 3: The Real Problem with MapReduce

MapReduce works, but programmers hated it.

Simple SQL Query:

```
SELECT SUM(sales) FROM orders;
```

In MapReduce Java, you need:

- Mapper class
- Reducer class
- Driver class
- Compilation
- JAR creation
- Cluster submission

Hundreds of lines — very painful. This led to ecosystem tools.

Chapter 4: Hadoop Ecosystem

Tools that sit on top of Hadoop:

- Hive
- Pig
- Sqoop
- Oozie

These tools compile into MapReduce jobs that run on HDFS.

Chapter 5: Hive

Created by: Facebook

Why Facebook Created Hive

Facebook had huge log data. Data analysts knew SQL but didn't know Java. Writing MapReduce for every query was impossible. Solution: allow SQL-like queries on Hadoop.

What you write:

```
SELECT * FROM customers;
```

Internal Flow:

```
Hive Query → Hive Compiler → MapReduce Job → HDFS → Result
```

Q: Does Hive process data itself?

A: No. Hive is only a query engine. Actual processing is done by MapReduce (or Spark/Tez in modern systems).

Why Hive is Called an Abstraction Layer

Because it hides complexity — users write simple SQL instead of hundreds of lines of Java code.

Chapter 6: Pig

Created by: Yahoo

Why Pig?

Yahoo engineers wanted something easier than Java but more flexible than SQL. They created Pig Latin.

Instead of 100 lines of MapReduce, you write:

```
A = LOAD 'sales';  
B = FILTER A BY amount > 1000;
```

Hive vs Pig

Aspect	Hive	Pig
Language	SQL-like	Scripting (Pig Latin)
Best For	Analysts	Developers
Data Model	Tables	Data flow
Creator	Facebook	Yahoo

Hive is preferred by SQL users and analysts. Pig is preferred by developers and data engineers.

Chapter 7: Sqoop

Sqoop = SQL + Hadoop

Big Question: How do we move data from MySQL to Hadoop?

Before Sqoop: Export CSV from MySQL → Copy file → Upload to HDFS. Very slow. Sqoop automates this process.

Import vs Export

Import (RDBMS → Hadoop): MySQL Customer Table → Sqoop → HDFS

Export (Hadoop → RDBMS): HDFS → Sqoop → Oracle

Important: Sqoop CAN export data, not just import. Common misconception: some say Sqoop only imports — False. Sqoop supports both import and export operations.

Q: Why is importing to HBase more common than exporting?

A: Because HBase is generally used as a destination storage system for large-scale NoSQL workloads. Data from relational databases is frequently loaded into HBase for faster distributed access. Exporting from HBase to RDBMS is less common because HBase has a flexible column-family model, while RDBMS requires a fixed relational schema, making conversion more complex.

Q: Can Sqoop transfer data directly between Hive and HBase?

A: No. Sqoop's main role is transferring bulk data between RDBMS and the Hadoop ecosystem. For Hive ↔ HBase transfers, Hadoop uses Hive-HBase integration, HBase APIs, Spark, or MapReduce jobs.

Chapter 8: Oozie

Created by: Yahoo

Purpose: Workflow Scheduling

Oozie is like Windows Task Scheduler or Cron Jobs for Hadoop.

Example Daily Pipeline:

2 AM → Import Data

3 AM → Clean Data

4 AM → Run Hive Query

5 AM → Generate Report

Oozie handles everything automatically.

Chapter 9: HBase

Key Difference from HDFS

HDFS is optimized for huge sequential reads. If you need customer id = 1001 quickly, HDFS is too slow.

What is HBase?

HBase is a NoSQL Database built on top of HDFS. HBase provides:

- Fast lookup
- Random read access
- Random write access

Real Example

Google Search needs milliseconds — cannot scan entire HDFS. Uses HBase-like systems for millisecond responses.

Chapter 10: Flume

Purpose: Collect streaming logs

Flume is a data ingestion tool.

Web Server Logs → Flume → HDFS

Real Example

Amazon receives clicks, user activity, and logs every second. Flume moves them into Hadoop in real-time.

Chapter 11: Mahout

Mahout = Machine Learning Library for Hadoop

Before Spark MLlib became popular, Mahout was used for:

- Recommendation systems
- Classification
- Clustering

Example: Netflix recommendations engine.

Chapter 12: Loosely Coupled Architecture

One of the most important interview concepts.

Tightly Coupled

Everything depends on everything. If one component fails, the entire system fails.

Loosely Coupled

Components are independent. HDFS, MapReduce, Hive, Sqoop, Pig each have separate roles and can be replaced.

Real Example

Earlier: Hive → MapReduce

Today: Hive → Spark

Hive still works — only the execution engine changed. This is possible because the architecture is loosely coupled.

Chapter 13: Metadata

Definition: Metadata = Data about data.

File contents: 1,1000 and 2,2000 → This is DATA

File metadata: Name, Size, Columns, Location, Owner, Created Date → This is METADATA

In Hive

Metadata is stored in the Metastore, containing: table names, column names, datatypes, partition info, HDFS locations.

Important: CREATE TABLE customer (id INT, name STRING) — Hive stores metadata. Actual data remains in HDFS.

Chapter 14: What Happens When You Run a Hive Query?

*Interview Question: SELECT * FROM customers;*

Hive Query

↓

Parser (syntax check)

↓

Compiler (converts to logical plan)

↓

Execution Plan

↓

MapReduce Job submission

↓

Data fetched from HDFS

↓

Result returned

Chapter 15: Why Hadoop Became Revolutionary

Before Hadoop: 1 Server stores and processes data — if server dies, everything dies

After Hadoop: 1000 machines store and process data — if one dies, system still works

This is called Fault Tolerance and Distributed Computing.

Key Interview Questions & Answers

Q: Why Hive when SQL already exists?

A: SQL needs structured data in RDBMS. Hive allows SQL-like queries on huge distributed data stored in HDFS.

Q: Why Pig if Hive already exists?

A: Hive is SQL oriented. Pig is scripting oriented. Programmers prefer Pig for flexibility.

Q: Why HBase when HDFS exists?

A: HDFS is for large sequential reads. HBase is for fast random reads/writes. Different use cases.

Q: Can Hadoop work without Hive?

A: Yes. Hive is optional. HDFS + processing engine (MapReduce/Spark) are core.

Q: Can Hive work without HDFS?

A: Traditionally no. Hive stores data in HDFS. Modern Hive can connect to other storage systems.

Q: Why is Hadoop called an ecosystem?

A: Because Hadoop is not one software — it is a collection of tools: HDFS, MapReduce, Hive, Pig, Sqoop, Oozie, HBase, Flume, Mahout, ZooKeeper, Spark, Kafka, and many more. Together they solve end-to-end big data problems.

Tool Classification

Interviewers expect this classification:

Category	Tools
Data Processing & Query Abstraction	Hive, Pig
Data Ingestion	Sqoop, Flume
Workflow Scheduling	Oozie
NoSQL Storage	HBase
Machine Learning	Mahout

LECTURE 3.1 — HADOOP FUNDAMENTALS

File System, Blocks, Client-Server, Distributed Systems

How is data physically stored on SSD/Hard Disk?

HDFS stores files as blocks, but the actual blocks are stored as normal files on the local Linux file system (Ext4/XFS) of each DataNode.

The Linux file system manages where those block files are physically placed on the HDD/SSD.

The hard disk or SSD ultimately stores the data as binary bits (0s and 1s) in magnetic sectors (HDD) or flash memory cells (SSD).

1. What is a File System?

A File System is software that manages how data is stored, organized, retrieved, and deleted on a storage device such as a hard disk or SSD.

Think of it as a manager between the Operating System and the Hard Disk.

Why do we need a File System?

Without a file system: files would have no names, the OS would not know where data is stored, and retrieving or deleting data would be almost impossible.

The file system keeps track of:

- File names
- File locations
- Folder structure
- Permissions
- Free space
- File size

Common File Systems

Operating System	File System
Windows	NTFS (New Technology File System)
Linux	Ext4 (earlier Ext2, Ext3)
macOS	APFS (Apple File System)

When Hadoop is installed on Linux, HDFS is stored on top of the Linux file system (usually Ext4), which in turn stores data on the physical hard disk.

Important Correction: HDFS is not a replacement for the Linux file system. HDFS uses the local Linux file system of each machine to store its data blocks.

2. File System vs Database

Many students confuse these two concepts.

File System

Stores files such as images, videos, PDFs, text files, music, programs. It does not enforce relationships between data — files are independent.

Database

A database stores structured data in rows and columns, providing searching, filtering, relationships, transactions, indexing, and security.

Very Important: Even databases eventually store data on the hard disk.

Application → Database (MySQL) → Operating System File System → Hard Disk

Database data ultimately reaches the hard disk through the operating system's file system.

3. What is a Block?

A Block is the smallest unit into which a file is divided before being stored.

Example: Suppose you have a file of 200 MB. If block size is 64 MB, then it is divided like this:

Block 1 = 64 MB

Block 2 = 64 MB

Block 3 = 64 MB

Block 4 = 8 MB

Notice that the last block does not have to be full.

Why use Blocks?

Instead of handling one huge file, the system handles many small pieces.

- Faster reading
- Faster writing
- Easier copying
- Easier recovery if one block is corrupted

Traditional File System	HDFS
All blocks stay on the same computer	Different blocks can be stored on different machines (DataNodes), allowing parallel access and fault tolerance

4. What is a Process?

A Process is simply a program that is currently running. When you open Chrome, VS Code, or Spotify, each becomes a separate process in memory.

Program = Stored on disk

Process = Program currently executing

5. What is a Daemon Process?

A Daemon is a background process that runs continuously without direct user interaction. It starts automatically when the system starts and waits to perform specific tasks.

Examples in Hadoop

- NameNode daemon
- DataNode daemon

- ResourceManager daemon

Examples in Linux

- SSH daemon
- Print daemon

6. What is a Cluster?

A Cluster is a collection of multiple computers connected together to work as a single system. Instead of one powerful computer, many computers share the workload.

- More storage
- Faster processing
- High availability
- Easy scalability

7. What is a Node?

Each individual computer inside a cluster is called a Node. One cluster consists of many nodes.

8. Types of Distributed Architecture

(A) Peer-to-Peer (P2P)

In a peer-to-peer system, every node has equal responsibility and can communicate directly with every other node.

Examples: BitTorrent, some blockchain networks.

Advantages: No central controller, no single point of failure. Disadvantages: Harder to manage, coordination is more complex.

(B) Master-Worker (Master-Slave)

One master node controls many worker (slave) nodes.

The Master: assigns work, tracks resources, maintains metadata. Workers: execute assigned tasks, store/process data.

Hadoop originally follows this architecture: NameNode (Master) and DataNodes (Workers).

Single Point of Failure (SPOF)

Earlier Hadoop versions had only one NameNode. If it failed: cluster stopped working, metadata became unavailable.

Modern Hadoop solves this using High Availability (Active and Standby NameNodes).

9. What is Client-Server Architecture?

Client requests a service. Server processes the request. Server sends back the response.

Examples: Gmail, YouTube, Banking apps.

10. What is a Client API?

An API (Application Programming Interface) is a set of rules that allows two software systems to communicate.

A Client API is the library or interface used by the client application to communicate with the server.

Real-Life Analogy

Imagine ordering food at a restaurant: You = Client, Waiter = API, Kitchen = Server.

You don't enter the kitchen — you tell the waiter what you need. The waiter communicates with the kitchen and brings back the food.

Similarly in Hadoop: Application (Client) uses API to communicate with Hadoop Services (Server).

Key Exam & Interview Points

Q: Why do we need a file system if we already have a hard disk?

A: Because the hard disk only stores raw bytes. The file system organizes those bytes into meaningful files and

directories.

Q: Why do we need a database if we already have a file system?

A: A file system stores files, while a database stores structured, searchable, and related data with advanced features like indexing, transactions, and concurrency.

Q: Why are blocks used?

A: Blocks make storage, retrieval, replication, and parallel processing much more efficient.

Q: What is the difference between a node and a cluster?

A: A node is one computer. A cluster is a group of connected nodes working together.

Q: Why does Hadoop use a master-worker architecture?

A: It simplifies coordination. The master manages metadata and task scheduling, while worker nodes focus on storing and processing data.

Revision Summary

Term	Definition
File System	Organizes files on a storage device
Database	Stores structured data with querying capabilities
Block	Smallest storage unit of a file
Process	A running program
Daemon	A background process running continuously
Node	One computer in a cluster
Cluster	Multiple connected computers working as one
Peer-to-Peer	All nodes are equal
Master-Worker	One master controls many workers
Client	Requests a service
Server	Provides the service
Client API	Interface used by applications to communicate with services such as HDFS

LECTURE 3.2 — HDFS

Why HDFS?

Traditional file systems work well for small amounts of data stored on a single machine. As data grows to terabytes or petabytes, a single machine becomes insufficient because of:

- Limited storage
- Limited processing power
- Hardware failures
- High cost of scaling

HDFS solves these problems by storing data across multiple computers (DataNodes) and providing fault tolerance through replication.

Position of HDFS

Application (Hive, Pig, Spark, etc.)



HDFS



Linux File System (Ext4/XFS)



Hard Disk / SSD

Important: HDFS is not a replacement for the Linux file system. HDFS stores its data on top of the local Linux file system of every DataNode.

HDFS Characteristics

- Distributed Storage
- Fault Tolerant
- Highly Scalable

- High Throughput
- Commodity Hardware
- Write Once, Read Many
- Stores very large files efficiently

NameNode (Master)

The NameNode is the master of HDFS. It stores only metadata. It never stores actual file data.

Responsibilities

- File names
- Directory structure
- Block IDs
- DataNode locations
- Replication information
- Permissions
- Block mapping

Runs as: Daemon Process

Important: Single Point of Failure in Hadoop 1.x.

DataNode (Slave)

Stores the actual data blocks.

Responsibilities

- Store blocks
- Read blocks
- Write blocks
- Delete blocks
- Send Heartbeats
- Send Block Reports

Runs as a daemon process.

Secondary NameNode

Many students misunderstand this.

Secondary NameNode is NOT

- ✗ Backup NameNode
- ✗ Standby NameNode
- ✗ Failover Node

Purpose: Checkpointing

Responsibilities

- Downloads FsImage
- Downloads EditLog
- Merges both
- Creates new checkpoint
- Sends updated FsImage back

- Clears/Rolls EditLog

Metadata

Metadata means: Data about data.

Example — File: student.csv

Metadata: File Name, File Size, Owner, Permissions, Block IDs, DataNode Locations.

Metadata is stored only in the NameNode.

Data Blocks

Files are divided into blocks. Example: 300 MB File, Default Hadoop 1.x Block Size = 64 MB

300 MB

↓

64 MB + 64 MB + 64 MB + 64 MB + 44 MB

Last block can be smaller.

Replication

Default Replication Factor = 3

Block A

↓

DN1 DN3 DN6

Benefits: Fault Tolerance, Data Availability, Recovery after failures.

HDFS File Write Process

Step 1

User copies file into Linux File System.

Step 2

User executes:

hdfs dfs -put

or

hadoop fs -put

Step 3

HDFS Client API is created. The client knows File size and Block size, and divides the file into blocks.

Step 4

Client contacts NameNode with the request: "Where should I store these blocks?"

Step 5

NameNode checks Healthy DataNodes, Available storage, Rack awareness, Replication factor — then returns metadata.

Block1 → DN1, DN2, DN5

Block2 → DN3, DN6, DN8

Step 6

Client creates a write pipeline:

Client

↓

DN1

↓

DN2

↓

DN5

Client writes block only to DN1. DN1 stores it and forwards to DN2. DN2 stores it and forwards to DN5. Replication happens automatically.

Step 7 — Acknowledgement

DN5

↓

DN2

↓
DN1
↓
Client

After ACK reaches the client, write is completed. NameNode metadata is already updated.

HDFS Read Process

- Client requests file.
- NameNode returns block locations.
- Client directly reads blocks from nearest DataNodes.
- NameNode does not participate in data transfer.

Why Client Writes Directly?

To prevent NameNode from becoming a bottleneck. NameNode handles only metadata. Data transfer occurs directly between Client and DataNodes.

Heartbeat

Purpose: Checks whether DataNode is alive.

DataNode → NameNode

Frequency: approximately every 3 seconds.

Contains: Health status, Disk status, Availability.

If heartbeat stops, NameNode marks node as dead.

Block Report

Purpose: Tells NameNode which blocks are stored.

Frequency: approximately every 1 hour.

Contains: Complete block list.

DN3 → Block1, Block5, Block10

Used to detect: Missing blocks, Corrupt blocks, Extra replicas.

FsImage

Snapshot of HDFS metadata. Contains: Files, Directories, Block Mapping, Permissions. Stored on disk.

EditLog

Stores every metadata change — Create file, Delete file, Rename file, Change permission.

EditLog continuously grows.

Checkpointing

Performed by Secondary NameNode.

FsImage + EditLog

↓
Secondary NameNode

↓
Merged FsImage

↓
NameNode

↓
EditLog cleared

Benefits: Faster startup, Smaller EditLog, Better metadata management.

Rack Awareness

A Rack is a collection of multiple servers connected to one network switch.

Rack 1: DN1, DN2, DN3

Rack 2: DN4, DN5, DN6

Instead of storing all replicas on DN1, DN2, DN3, HDFS stores:

Replica1 → Rack1

Replica2 → Rack1

Replica3 → Rack2

Benefits: Survives rack failure, Better network utilization, High availability.

DataNode Failure

Hardware Failure

Machine permanently damaged.

- NameNode detects failure
- Missing replicas identified
- New replicas created on healthy DataNodes
- Metadata automatically updated
- Administrator replaces machine later

Software / Network Failure

Temporary downtime — heartbeat stops, node marked dead, replication begins.

If node returns, some blocks become over-replicated. NameNode automatically removes extra replicas. Administrator usually does not manually edit metadata.

Hadoop 1.x Limitation

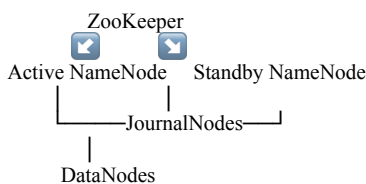
Only one NameNode.

NameNode → Fails → Entire Cluster Stops

This is called Single Point of Failure (SPOF).

Hadoop 2.x High Availability (HA)

Introduced to eliminate Single Point of Failure.



Components

Active NameNode

Handles Client Requests, Metadata, Block Management. Only one Active NameNode exists.

Standby NameNode

Continuously synchronized. Does not serve clients. Immediately becomes Active during failure. Also performs checkpointing.

JournalNode

Stores shared EditLogs. Usually 3 JournalNodes.

Active NameNode → JournalNodes → Standby NameNode

Standby continuously reads latest edits.

ZooKeeper

Coordination Service. Responsibilities:

- Detect Active NameNode failure
- Elect new Active NameNode

- Prevent split brain
- Automatic failover

Usually 3 or 5 ZooKeeper servers. Uses Leader-Follower architecture internally.

Automatic Failover

Active NameNode
↓
Fails
↓
ZooKeeper detects
↓
Standby promoted
↓
Clients reconnect
↓
Cluster continues

No manual intervention required.

Hadoop Version Comparison

Hadoop 1.x	Hadoop 2.x
One NameNode	Active + Standby NameNode
Secondary NameNode	Checkpointing + Standby performs synchronization
JobTracker	ResourceManager
TaskTracker	NodeManager
Single Point of Failure	High Availability
Manual recovery	Automatic failover

Daemons in Hadoop

Hadoop 1.x	Hadoop 2.x
Master: NameNode, JobTracker	Master: Active NameNode, Standby NameNode, ResourceManager
Slave: DataNode, TaskTracker	Slave: DataNode, NodeManager
Secondary: Secondary NameNode	Coordination: ZooKeeper, JournalNode

Important Interview Questions

Q: Does NameNode store actual data?

A: No. It stores only metadata.

Q: Who writes data to DataNodes?

A: The HDFS Client API writes directly to the DataNodes.

Q: Does NameNode split files?

A: No. The HDFS Client API divides the file into blocks based on the configured block size.

Q: What is stored in metadata?

A: File names, Block IDs, Block locations, Permissions, Replication information.

Difference between Heartbeat and Block Report?

Heartbeat	Block Report
Every ~3 sec	Every ~1 hour
Node health	Complete list of blocks
Detect dead nodes	Detect missing/corrupt blocks

Q: Why is Secondary NameNode not a backup?

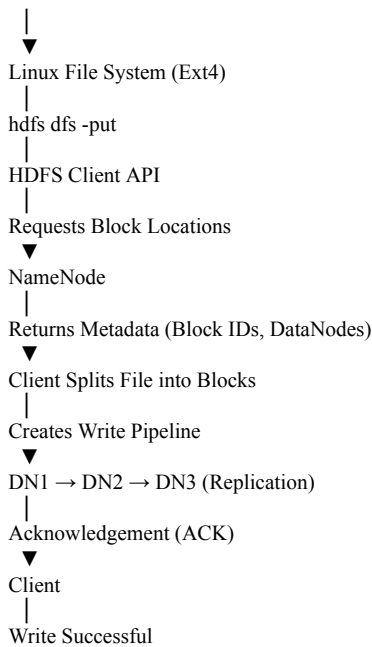
A: Because it only merges FsImage and EditLog to create checkpoints. It cannot immediately replace a failed NameNode.

Q: What happens if a DataNode returns after replication has already occurred?

A: The NameNode detects over-replicated blocks and automatically removes the extra replicas to maintain the configured replication factor.

Complete End-to-End HDFS Flow

User



Meanwhile...

DataNode —Heartbeat (~3 sec)→ NameNode
DataNode —Block Report (~1 hr)→ NameNode

NameNode

- FsImage (Metadata Snapshot)
- EditLog (Recent Changes)
- Secondary NameNode periodically merges them into a new checkpoint.

If NameNode fails (Hadoop 2.x):
Active NameNode → JournalNodes → Standby NameNode
ZooKeeper detects failure → Standby promoted to Active
Clients reconnect automatically.

How is data physically stored on SSD/Hard Disk?

HDFS stores files as blocks, but the actual blocks are stored as normal files on the local Linux file system (Ext4/XFS) of each DataNode. The Linux file system manages where those block files are physically placed on the HDD/SSD.

The hard disk or SSD ultimately stores the data as binary bits (0s and 1s) in magnetic sectors (HDD) or flash memory cells (SSD).

How does checkpointing happen in Hadoop 2.x?

The Standby NameNode continuously reads EditLogs from the JournalNodes and updates its own metadata.

The Standby NameNode periodically performs checkpointing by merging the latest FsImage and EditLogs into a new FsImage.

Unlike Hadoop 1.x, a separate Secondary NameNode is not required for High Availability.

What happens if a DataNode fails while writing a block?

If a DataNode fails during the write pipeline, the pipeline breaks and the client detects the failure.

The client reports the failed DataNode to the NameNode. The NameNode selects a new healthy DataNode for the missing replica.

The client rebuilds the pipeline and continues writing from the last successful replica (it does not restart the entire file upload). After successful replication, the NameNode updates the metadata automatically.

Correction: The ACK is not sent because the node failed. Instead, the client notices that the expected ACK from the failed DataNode does not arrive (or a write error occurs), then contacts the NameNode for a new DataNode.

One More Useful Interview Line

Q: Who stores metadata?

A: Only the NameNode stores metadata; DataNodes store only the actual data blocks.

Q: Who updates metadata?

A: Metadata is updated automatically by the NameNode whenever blocks are created, deleted, replicated, or moved.

Q: Does the NameNode receive file data?

A: No. The NameNode receives only metadata requests; the actual file data flows directly between the client and DataNodes.

LECTURE 3.3 — MAPREDUCE

Hadoop 1.x MapReduce Architecture

Definition

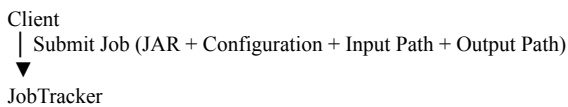
MapReduce is a distributed programming model used to process huge datasets stored in HDFS by dividing the work into smaller tasks (Map and Reduce) that execute in parallel across multiple DataNodes.

Main Components

Component	Role
Client	Submits MapReduce Job (JAR + Configuration)
JobTracker	Master daemon — receives job, contacts NameNode, schedules tasks, monitors TaskTrackers, handles failures
TaskTracker	Slave daemon on every DataNode — executes Map/Reduce tasks, launches JVM per task, sends heartbeat to JobTracker
NameNode	Stores metadata — block locations, file namespace
DataNode	Stores actual HDFS blocks
Mapper	Processes input records
Reducer	Aggregates Mapper output

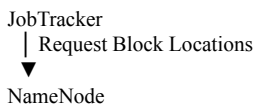
Complete Workflow

Step 1 — Job Submission



The client submits only the program and configuration. The input data is already stored in HDFS.

Step 2 — Metadata Request



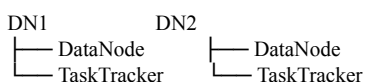
NameNode returns:

- Block-1 → DN1
- Block-2 → DN4
- Block-3 → DN2

Only metadata is returned. The NameNode never transfers data.

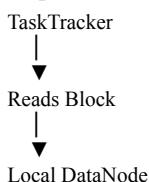
Step 3 — Scheduling (Data Locality)

JobTracker chooses the TaskTracker located on the same DataNode whenever possible.



This is called Data Locality — moving computation to data is much faster than moving data across the network.

Step 4 — Reading Data



Since both are on the same machine, network traffic is minimized.

Step 5 — JVM Creation

TaskTracker launches a JVM for every running task.

Example — Task Slots = 2, Blocks = Block1, Block2, Block3

JVM1 → Block1
JVM2 → Block2
Block3 → Waiting

After JVM1 finishes:

JVM1 → Block3

Step 6 — Mapping

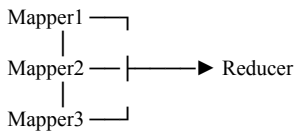
Mapper processes one record at a time.

Input:	Mapper Output:
Apple	<Apple,1>
Banana	<Banana,1>
Apple	<Apple,1>

Intermediate output is written to the Local File System — NOT HDFS.

Step 7 — Shuffle

All Mapper outputs are transferred through HTTP.



Step 8 — Sort

Before the Reducer starts, the framework automatically sorts keys.

Before:	After:
Apple	Apple
Banana	Apple
Apple	Banana
Orange	Orange

Same keys are grouped together.

Step 9 — Reduce

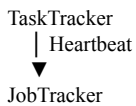
Apple → [1,1]	Output:
Banana → [1]	Apple → 2
	Banana → 1

Step 10 — Output

Reducer writes the final result into HDFS. This becomes the final output directory.

Fault Tolerance

Every TaskTracker continuously sends a Heartbeat to the JobTracker.



If the heartbeat stops, JobTracker assumes Node Failure and contacts the NameNode again.

NameNode → Block-1
Replica 1 → DN1
Replica 2 → DN5
Replica 3 → DN8

JobTracker reschedules the task on another healthy TaskTracker. No manual intervention is required.

Mapper Output Flow

Mapper Output → Intermediate Data → Stored in Local Disk → Shuffle → Reducer → HDFS

Q: Why isn't Mapper output stored in HDFS?

A: Because it is temporary data. Writing every Mapper output to HDFS would create unnecessary disk I/O and replication overhead, making MapReduce much slower.

Why MapReduce?

- Parallel Processing
- Fault Tolerance
- Automatic Task Scheduling
- Data Locality
- Horizontal Scalability
- Handles TB/PB Data
- Automatic Recovery
- Distributed Computing

Advantages

- ✓ Scalable
- ✓ Reliable
- ✓ Cost Effective
- ✓ Processes Huge Files
- ✓ Automatic Recovery
- ✓ Uses Commodity Hardware
- ✓ Parallel Execution
- ✓ Easy to Scale by Adding Nodes

Limitations

- ✗ JobTracker is Single Point of Failure
- ✗ High Disk I/O
- ✗ Batch Processing Only
- ✗ Slow for Real-Time Analytics
- ✗ Every Phase writes to Disk
- ✗ Fixed Map/Reduce Slots

Hadoop 2.x (YARN) Architecture

Why YARN?

Problem with Hadoop 1.x

JobTracker handled everything:

- Resource Management
- Job Scheduling
- Task Monitoring
- Failure Recovery

Problems:

- ✗ Single Point of Failure
- ✗ Scalability Bottleneck
- ✗ Could run only MapReduce jobs

Solution

Hadoop 2.x introduced YARN (Yet Another Resource Negotiator).

Purpose: Separate Resource Management from Job Execution.

Main Components

Component	Role
Client	Submits Application (JAR + Configuration)
ResourceManager (RM)	Master daemon — allocates cluster resources, schedules Containers, receives heartbeats from NodeManagers
NodeManager (NM)	Slave daemon on every worker node — manages Containers, monitors CPU & Memory, launches Containers, sends heartbeats to RM

ApplicationMaster (AM)	One per application — communicates with RM, requests Containers, monitors Map & Reduce tasks, restarts failed tasks
Container	Smallest unit of execution — contains CPU + RAM, runs one task inside a JVM
NameNode	Metadata
DataNode	Stores HDFS Blocks

Complete Workflow

Step 1 — Job Submission

Client

| Submit Job (JAR + Config + Input Path + Output Path)



ResourceManager

Unlike Hadoop 1.x, the job is NOT sent to JobTracker — it is submitted to the ResourceManager.

Step 2 — Launch ApplicationMaster

ResourceManager allocates one Container to start the ApplicationMaster.

Client → ResourceManager → Container → ApplicationMaster

Think of the ApplicationMaster as the manager of only this one application.

Step 3 — Metadata Request

ApplicationMaster → NameNode → Block Locations

Again, NameNode returns only Metadata — NOT Actual Data.

Step 4 — Request Containers

ApplicationMaster now knows:

- Block1 → Node1
- Block2 → Node5
- Block3 → Node2

It asks ResourceManager: "I need 3 Containers."

Step 5 — Allocate Containers

ResourceManager checks Available CPU, Available RAM, Available Nodes — then allocates Containers.

Node1 → Container1

Node2 → Container2

Node5 → Container3

ResourceManager always tries Data Locality — run the task where the data already exists.

Step 6 — NodeManager Starts Containers

Every allocated NodeManager launches:

Container → JVM → Map Task

Unlike Hadoop 1.x, NodeManager launches Containers, not individual tasks directly.

Step 7 — Mapper Reads Data

Container reads block from Local DataNode.

Node5: DataNode → Container → Mapper

Again, moving computation is cheaper than moving data.

Step 8 — Mapper Output

Input: Mapper Output:

Apple <Apple,1>

Banana <Banana,1>

Apple <Apple,1>

Stored in Local Disk — NOT HDFS.

Step 9 — Shuffle

Mapper1, Mapper2, Mapper3 → Reducer (Protocol: HTTP)

Step 10 — Sort

Framework automatically sorts keys and groups same keys.

Apple

Apple
Banana
Orange

Step 11 — Reduce

Apple → [1,1] Output:
Banana → [1] Apple → 2
 Banana → 1

Step 12 — Write Output

Reducer writes Final Output → HDFS

Step 13 — Job Completion

ApplicationMaster reports Job Completed to ResourceManager. ApplicationMaster exits. Containers are released. Resources become available for new jobs.

Fault Tolerance

Heartbeats

Every NodeManager → Heartbeat → ResourceManager

If heartbeat stops, RM knows Node Failure.

Container Failure

Suppose Container2 fails. ApplicationMaster notices the failure and requests another Container.

ApplicationMaster → ResourceManager → New Container → Restart Task

Only the failed task restarts. Entire job is not restarted.

Data Locality

HDFS Block → Same Node → Container → Mapper

Goal: Move Computation, NOT Data.

Container

Container is NOT a Docker container. It is simply Allocated Resources:

2 CPU + 4 GB RAM + JVM = Container → Runs Map Task

Hadoop 1.x vs Hadoop 2.x

Hadoop 1.x	Hadoop 2.x (YARN)
JobTracker	ResourceManager + ApplicationMaster
TaskTracker	NodeManager
Fixed Map/Reduce Slots	Dynamic Containers
Single Point of Failure	Better fault tolerance
MapReduce only	Supports MapReduce, Spark, Hive, Tez, Flink, etc.
Poor scalability	Highly scalable

Interview Questions

Q: Why was YARN introduced?

A: To remove the JobTracker bottleneck by separating Resource Management, Job Scheduling, and Job Execution.

Q: Who launches the ApplicationMaster?

A: ResourceManager

Q: Who launches Containers?

A: NodeManager

Q: Who requests Containers?

A: ApplicationMaster

Q: Who allocates Containers?

A: ResourceManager

Q: Who monitors Tasks?

A: ApplicationMaster

Q: Who monitors Nodes?

A: ResourceManager

Q: Who stores Metadata?**A:** NameNode**Q: Who stores Actual Data?****A:** DataNode**Important Correction***You mentioned: "YARN came as an extension of JobTracker." — This is not technically correct.***The correct statement is:**

Hadoop 1.x

↓

JobTracker (Resource Management + Scheduling + Monitoring)

↓ Problems

- Single Point of Failure
- Scalability Issues
- Only MapReduce Supported

↓ Solution

Hadoop 2.x introduced YARN.

JobTracker was split into:

- ResourceManager (Cluster Resource Management)
- ApplicationMaster (Per-Application Management)

TaskTracker was replaced by:

- NodeManager

Execution now happens inside Containers.

This is the explanation expected in interviews.

Key-Value Pair, InputFormat & RecordReader

Key-Value Pair in MapReduce

Definition

MapReduce processes every piece of data as a <Key, Value> pair. Both the Mapper input/output and Reducer input/output are key-value pairs.

Why Key-Value Pairs?

- Enables parallel processing
- Makes sorting easy
- Makes grouping easy
- Allows Reducer to aggregate same keys

The framework uses the key to decide which Reducer should process the data.

Example — Word Count

Input File:	Mapper Output:	Shuffle & Sort:	Reducer Output:
Apple	<Apple,1>	Apple → [1,1]	<Apple,2>
Banana	<Banana,1>	Banana → [1]	<Banana,1>
Apple	<Apple,1>	Orange → [1]	<Orange,1>
Orange	<Orange,1>		

MapReduce Data Flow

Input File → InputFormat → Mapper → <Key,Value> → Shuffle → Sort → Reducer → <Key,Value> → OutputFormat → HDFS

InputFormat

Definition

InputFormat decides how input files are divided into InputSplits, which RecordReader is used, and how records

are sent to the Mapper. Think of it as the bridge between HDFS and the Mapper.

Responsibilities

- 1. Split Input File
- 2. Create RecordReader
- 3. Convert records into <Key,Value>
- 4. Send records to Mapper

InputSplit

InputSplit is NOT the actual HDFS block.

HDFS Block → Logical InputSplit → Mapper

HDFS Block	InputSplit
Physical storage unit	Logical processing unit

Normally, 1 HDFS Block $\approx$ 1 InputSplit — but they are not exactly the same.

RecordReader

RecordReader converts the InputSplit into key-value records.

InputSplit → RecordReader → <Key,Value> → Mapper

Without the RecordReader, the Mapper cannot understand the input.

TextInputFormat (Default)

This is the most commonly used InputFormat. Each line becomes one record.

Input File:	Mapper Receives:
Hello Hadoop	<0,"Hello Hadoop">
Hello Spark	<13,"Hello Spark">
Hello Hive	<25,"Hello Hive">

Meaning — Key = Byte Offset (starting position of line); Value = Entire Line

What is Byte Offset?

Suppose the file is: "Hello Hadoop" / "Hello Spark". Counting bytes (ASCII):

Hello = 5, Space = 1, Hadoop = 6, Newline = 1 → Total = 13 bytes

Line 1 → Offset = 0

Line 2 → Offset = 13

So the Mapper receives:

<0,"Hello Hadoop">
<13,"Hello Spark">

Q: Why is the offset used as the key?

A: The offset uniquely identifies each line in the file. It is mainly used by the framework to keep track of records. Most applications ignore it.

KeyValueTextInputFormat

Used when the file already contains key-value data.

Input File (Separator = ","):	Mapper Receives:
101,John	<101,John>
102,Ram	<102,Ram>
103,David	<103,David>

Everything before the separator becomes the key, and everything after it becomes the value.

TextOutputFormat (Default)

Reducer writes the output as Key<TAB>Value:

Apple	2
Banana	1
Orange	1

The key and value are separated by a tab.

Common InputFormats

InputFormat	Description
TextInputFormat	Default — Key = Byte Offset, Value = Entire Line
KeyValueTextInputFormat	Reads existing key-value files
SequenceFileInputFormat	Reads Hadoop binary SequenceFiles
NLineInputFormat	Sends N lines to one Mapper

Interview Summary

Term	Meaning
Key-Value Pair	Basic data structure used by MapReduce
InputFormat	Splits input and creates RecordReader
RecordReader	Converts InputSplit into <Key, Value>
TextInputFormat	Key = Byte Offset, Value = Entire Line
KeyValueTextInputFormat	Reads existing key-value records
Mapper Output	Intermediate <Key, Value>
Shuffle	Transfers Mapper output to Reducer
Sort	Groups identical keys
Reducer Output	Final <Key, Value>
TextOutputFormat	Writes Key<TAB>Value into HDFS

Important Correction

Many students say: "The key in TextInputFormat is the line number." — ❌ Wrong.

The key is the byte offset (starting byte position of the line), not the line number.

File:
Hello Hadoop
Hello Spark
Hello Hive

Mapper Input:
<0, "Hello Hadoop">
<13, "Hello Spark">
<25, "Hello Hive">

Here 0, 13, and 25 are byte offsets, not line numbers.

This is one of the most frequently asked Hadoop interview questions.

LECTURE 4 — HDFS INSTALLATION AND COMMANDS

For a Data Engineer / Hadoop Administrator interview, you don't need all 200+ HDFS commands. Around 50-60 commands cover almost everything asked in interviews and day-to-day work.

1. File & Directory Commands ★★★★★

Command	Purpose
hdfs dfs -ls /	List files/directories
hdfs dfs -mkdir /data	Create directory
hdfs dfs -mkdir -p /a/b/c	Create nested directories
hdfs dfs -put file.txt /data/	Upload local → HDFS
hdfs dfs -copyFromLocal file.txt /data/	Upload file
hdfs dfs -get /data/file.txt .	Download HDFS → Local
hdfs dfs -copyToLocal /data/file.txt .	Download file
hdfs dfs -cat /file	Display file
hdfs dfs -tail /file	Last few lines
hdfs dfs -head /file	First few lines
hdfs dfs -du -h /	Directory size
hdfs dfs -df -h	HDFS free space
hdfs dfs -count /dir	Count files/directories
hdfs dfs -find / -name "*.csv"	Find files

2. Copy & Move Commands ★★★★★

Command	Purpose
hdfs dfs -cp	Copy files
hdfs dfs -mv	Move/Rename
hdfs dfs -rm file	Delete file
hdfs dfs -rm -r dir	Delete directory
hdfs dfs -rmdir dir	Remove empty directory
hdfs dfs -expunge	Empty Trash

3. Permissions & Ownership ★★★★★

Command	Purpose
hdfs dfs -chmod 755 file	Change permission
hdfs dfs -chown user:group file	Change owner
hdfs dfs -chgrp group file	Change group
hdfs dfs -getfacl file	View ACL
hdfs dfs -setfacl	Set ACL

4. File Information ★★★★★

Command	Purpose
hdfs dfs -stat file	File metadata
hdfs fsck /	Check filesystem health
hdfs dfs -checksum file	Check checksum
hdfs dfs -test -e file	Check existence
hdfs dfs -test -d dir	Check directory
hdfs dfs -test -z file	Check empty file

5. Replication Commands ★★★★★

Command	Purpose
hdfs dfs -setrep 3 file	Change replication
hdfs dfs -setrep -w 3 file	Wait until replication completes

6. Quota Commands (Very Important Admin) ★★★★★

Namespace Quota (Number of files)

hdfs dfsadmin -setQuota 1000 /data

Maximum 1000 files/directories.

Remove:

hdfs dfsadmin -clrQuota /data

Space Quota (Storage)

hdfs dfsadmin -setSpaceQuota 100G /data

Maximum 100 GB.

Remove:

hdfs dfsadmin -clrSpaceQuota /data

Check:

hdfs dfs -count -q /data

7. Cluster Admin Commands ★★★★★

Command	Purpose
hdfs dfsadmin -report	Cluster report
hdfs dfsadmin -safemode get	Check Safe Mode
hdfs dfsadmin -safemode leave	Exit Safe Mode
hdfs dfsadmin -safemode enter	Enter Safe Mode
hdfs dfsadmin -refreshNodes	Refresh DataNodes
hdfs dfsadmin -printTopology	Rack awareness
hdfs dfsadmin -refreshUserToGroupsMappings	Refresh users/groups

8. Filesystem Check (FSCK) ★★★★★

hdfs fsck /

Check filesystem.

hdfs fsck / -files

List files.

hdfs fsck / -blocks

Show blocks.

hdfs fsck / -locations

Show block locations.

hdfs fsck / -move

Move corrupted files.

hdfs fsck / -delete

Delete corrupted files.

9. Balancer Commands ★★★★★

hdfs balancer

Balance storage.

hdfs balancer -threshold 10

Balance if usage differs by more than 10%.

10. NameNode Commands ★★★★★

hdfs namenode -format

Format NameNode (initial setup only).

hdfs namenode -metadataVersion

Metadata version.

hdfs namenode -recover

Recover metadata.

11. Secondary/Checkpoint Commands ★★★

hdfs secondarynamenode

Start Secondary NameNode.

hdfs secondarynamenode -checkpoint

Trigger checkpoint.

12. DataNode Commands ★★★

hdfs datanode

Start DataNode.

13. HA Commands ★★★★★

`hdfs haadmin -getServiceState nn1`

Check active/standby.

`hdfs haadmin -failover nn1 nn2`

Manual failover.

14. Snapshot Commands ★★★★★

`hdfs dfsadmin -allowSnapshot /data`

Enable snapshots.

`hdfs dfs -createSnapshot /data snap1`

Create snapshot.

`hdfs dfs -renameSnapshot /data snap1 snap2`

Rename snapshot.

`hdfs dfs -deleteSnapshot /data snap2`

Delete snapshot.

15. Storage Policy Commands ★★★

`hdfs storagepolicies -listPolicies`

List policies.

`hdfs storagepolicies -getStoragePolicy /data`

View policy.

`hdfs storagepolicies -setStoragePolicy -path /data -policy COLD`

Set storage policy.

16. Most Asked Interview Commands ★★★★★

`hdfs dfs -ls`
`hdfs dfs -put`
`hdfs dfs -get`
`hdfs dfs -cat`
`hdfs dfs -tail`
`hdfs dfs -mkdir`
`hdfs dfs -rm -r`
`hdfs dfs -mv`
`hdfs dfs -cp`
`hdfs dfs -du`
`hdfs dfs -count`
`hdfs dfs -chmod`
`hdfs dfs -chown`
`hdfs dfs -setrep`
`hdfs dfsadmin -report`
`hdfs dfsadmin -safemode get`
`hdfs fsck /`
`hdfs balancer`
`hdfs dfsadmin -setQuota`
`hdfs dfsadmin -setSpaceQuota`
`hdfs haadmin -failover`

LECTURE 5 — HIVE

A. APACHE HIVE — INTRODUCTION & ARCHITECTURE

What is Hive?

Apache Hive is an open-source Data Warehouse and SQL Query Engine built on top of Hadoop that allows users to analyze and query large datasets stored in HDFS using HiveQL (SQL-like language) instead of writing Java MapReduce programs.

Definition (Interview): Hive is a Data Warehouse infrastructure built on Hadoop that provides an SQL

abstraction layer (HiveQL) to process and analyze large datasets stored in HDFS.

Why was Hive Developed?

Initially, Hadoop supported data processing only through MapReduce, which required writing Java programs.

Without Hive:

```
graph TD
    User --> Write[Write Java MapReduce Program]
    Write --> Compile[Compile]
    Compile --> Run[Run]
    Run --> Output[Output]
```

Since most analysts at Facebook knew SQL but not Java, Facebook developed Hive.

Now users can simply write:

```
SELECT * FROM employee;
```

instead of hundreds of lines of Java code.

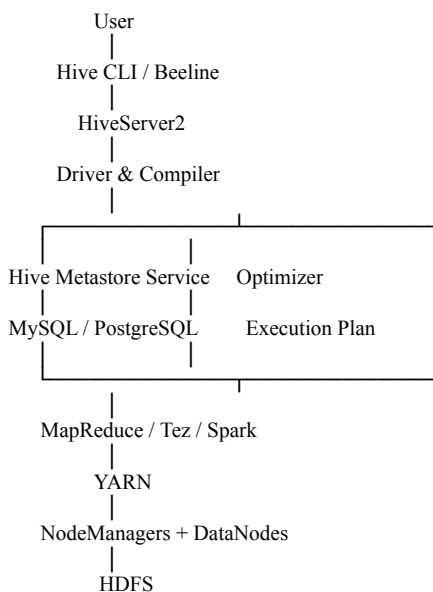
Is Hive a Database?

No.

Hive is NOT a database. Hive is a Query Engine, a Data Warehouse, and an SQL Abstraction Layer.

Actual data is stored in HDFS. Hive only provides SQL access.

Hive Architecture



Components

1. Hive Client

User writes HiveQL.

```
SELECT * FROM employee;
```

2. HiveServer2

Receives user queries. Supports Multiple users, JDBC, ODBC, Beeline.

3. Driver

Responsible for managing query execution, calling compiler, calling optimizer, returning results.

4. Compiler

Checks Syntax, Semantic errors, Table existence, Column names.

Creates a Logical Plan.

5. Optimizer

Optimizes query — e.g. Predicate Pushdown, Partition Pruning, Join Optimization.

Produces a Physical Execution Plan.

6. Execution Engine

Executes query using MapReduce, Tez, or Spark.

7. Hive Metastore

Stores Table Name, Database Name, Column Names, Datatypes, Partitions, Buckets, Owner, Statistics, HDFS Location, Table Properties.

Important: Metastore NEVER stores actual data. Only metadata.

Metadata

Metadata means: Data about Data.

Example — Table: employee

Columns: id, name, salary, dept

Datatype: INT, STRING, DOUBLE, STRING

Location: /user/hive/warehouse/employee

Owner: admin

Partitions: city

All these are metadata.

Where is Metadata Stored?

Inside the Hive Metastore. Production uses MySQL, PostgreSQL, or Oracle.

Embedded Derby vs Remote Metastore

Embedded Derby (Default)

HiveServer1

|

Derby

Suitable only for Learning and Testing — not production.

Problem with Embedded Derby

Suppose HiveServer1 creates:

CREATE TABLE employee(...);

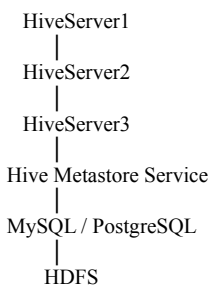
Metadata stored in HiveServer1's Derby. Now another query goes to HiveServer2:

SELECT * FROM employee;

HiveServer2 checks its own Derby. Nothing exists. Result: Table not found.

Each embedded Derby has its own independent metadata.

Remote Metastore (Production)



Every HiveServer reads the same metadata.

Advantages of Remote Metastore

- ✓ Shared metadata across all HiveServers
- ✓ Multiple concurrent users
- ✓ High Availability
- ✓ Easy backup
- ✓ Better performance
- ✓ Centralized management

Can be shared with Spark SQL, Impala, Presto, Trino.

Where is Data Stored?

Actual data is stored inside HDFS. By default: /user/hive/warehouse/

/user/hive/warehouse/

employee/

part-00000
part-00001
part-00002

These are actual HDFS files. HDFS knows only files — it knows nothing about tables.

How Hive Knows Data Location?

Metastore stores:

employee
↓
Location
/user/hive/warehouse/employee

Whenever `SELECT * FROM employee;` runs, Hive asks Metastore, which replies with the location. Hive then requests HDFS to give files from that location. HDFS returns file blocks and execution starts.

Complete Query Execution Flow

Suppose:

`SELECT *`
`FROM employee`
`WHERE dept='IT';`

User
↓
HiveServer2
↓
Driver
↓
Compiler
↓
Syntax Check
↓
Semantic Check
↓
Hive Metastore
↓
Get Table Metadata
↓
HDFS Location
↓
Execution Plan
↓
MapReduce / Spark / Tez
↓
YARN
↓
Mapper
↓
Reducer (if required)
↓
Output

Does Hive Convert SQL into Java Code?

No.

Hive never generates Java source code. Instead:

HiveQL
↓
Parser
↓
Logical Plan
↓
Optimizer
↓
Physical Plan
↓
Uses Hadoop Java APIs
↓
Creates MapReduce Job
↓
Submits Job to YARN

Hive internally calls Hadoop's Java APIs like `Job job = Job.getInstance(conf);` to create and submit MapReduce jobs. It does NOT generate .java files.

When Does MapReduce Run?

Query — `SELECT * FROM employee;`

Usually no MapReduce job.

Filter — `SELECT * FROM employee WHERE dept='IT';`

Usually no MapReduce job (simple scan).

Aggregation — `SELECT dept, COUNT(*) FROM employee GROUP BY dept;`

Execution engine required.

Join — `SELECT * FROM emp JOIN dept ON emp.id=dept.id;`

Execution engine required.

Order By — `ORDER BY salary;`

Execution engine required.

Creating Database

`CREATE DATABASE company;`

`SHOW DATABASES;`

`USE company;`

`DROP DATABASE company;`

Creating Table

`CREATE TABLE employee(`

`id INT,`

`name STRING,`

`salary DOUBLE,`

`dept STRING`

`);`

Show tables:

`SHOW TABLES;`

Describe table:

`DESCRIBE employee;`

Detailed description:

`DESCRIBE FORMATTED employee;`

DDL statement:

`SHOW CREATE TABLE employee;`

Default Storage

Creating `CREATE TABLE employee(...)` creates metadata pointing to `/user/hive/warehouse/employee`. Initially, `employee/` is empty.

Loading Data

1. Local File

`LOAD DATA LOCAL INPATH '/home/data.csv'`

`INTO TABLE employee;`

Copies local file.

2. HDFS File

`LOAD DATA INPATH '/input/data.csv'`

`INTO TABLE employee;`

Moves/copies HDFS file. Usually no MapReduce job.

3. Insert

`INSERT INTO employee`

`VALUES`

`(1,'Ram',50000,'IT');`

Runs execution engine.

Important File Behavior (Interview)

Suppose employee/000000_0 contains:

```
1,Ram,IT
2,John,HR
```

Now execute:

```
INSERT INTO employee
VALUES
(3,'David','Sales');
```

Hive does not modify the existing file. Instead, it creates a new file.

Before:

```
employee/
000000_0
```

After INSERT:

```
employee/
000000_0
000001_0
```

Another INSERT:

```
employee/
000000_0
000001_0
000002_0
```

Interview Point

- ✓ Every successful INSERT generally creates new data files rather than modifying existing files.
- ✓ Over time, a table may contain hundreds or thousands of small files, which can reduce performance (the "small files problem").
- ✓ Modern Hive ACID tables support updates and deletes through delta files and periodic compaction, but the principle remains that Hive prefers append-style writes rather than in-place modification.

Managed (Internal) Table

Hive manages both Metadata and Data.

```
CREATE TABLE employee(
  id INT,
  name STRING
);
```

Stored by default in /user/hive/warehouse/

```
DROP TABLE employee;
```

Deletes: Metadata AND HDFS Data.

External Table

Hive manages only metadata.

```
CREATE EXTERNAL TABLE employee(
  id INT,
  name STRING
)
LOCATION '/project/data/';
```

```
DROP TABLE employee;
```

Deletes: Metadata only. Keeps: HDFS Data.

Managed vs External

Feature	Managed Table	External Table
Data Ownership	Hive	User/Application
Metadata	Deleted	Deleted
Data	Deleted	Remains
Default Location	/user/hive/warehouse/	User-specified or external path
Best For	Hive-only data	Shared datasets (Spark, Hive, Trino, Impala, etc.)

ACID Tables

Important Interview Question — not every managed table is ACID.

Correct relationship:

ACID Table
↓
Managed Table
↓
Hive Table

All ACID tables are managed tables. Not all managed tables are ACID.

ACID requires Transactional properties, a supported storage format (typically ORC), and appropriate Hive configuration.

Important Interview Questions

Q: Is Hive a database?

A: No. It is a Data Warehouse and SQL Query Engine.

Q: Where is data stored?

A: In HDFS.

Q: Where is metadata stored?

A: Hive Metastore (MySQL/PostgreSQL/Oracle in production).

Q: Why is MySQL preferred over Derby?

A: Shared metadata, multiple users, better scalability, high availability, backup & recovery, and it can be shared by multiple query engines.

Q: Does Hive convert SQL into Java?

A: No. Hive creates an execution plan and uses Hadoop's Java APIs to configure and submit MapReduce, Tez, or Spark jobs.

Q: Why does Hive create new files during INSERT?

A: HDFS is optimized for write-once, read-many workloads. Instead of modifying existing files, Hive appends data by creating new files. This design provides scalability and fault tolerance but can lead to many small files over time.

Complete Hive Workflow (One-Line Summary)

User → HiveServer2 → Driver → Compiler → Optimizer → Hive Metastore → Get HDFS Location → MapReduce/Tez/Spark → YARN → DataNodes (HDFS Files) → Result to User

B. HIVE PARTITIONING

What is Partitioning?

Partitioning is a performance optimization technique that physically divides a Hive table into multiple directories based on the value of one or more partition columns.

Instead of scanning the entire table, Hive scans only the required partition(s), reducing I/O and improving query performance.

Definition (Interview): Partitioning is the process of dividing a large Hive table into smaller logical parts (directories in HDFS) based on partition column values, allowing Hive to scan only the relevant data instead of the entire table.

Why Partitioning?

Suppose we have 100 crore (1 billion) sales records.

Without partition:

sales

part-00000
part-00001
part-00002
part-00003
...

SELECT *
FROM sales
WHERE city='Chennai';

Hive has no idea where Chennai records are.

✗ Reads every file

- ✗ Full Table Scan
- ✗ Slow

With partition:

sales

city=Chennai/
city=Bangalore/
city=Delhi/
city=Mumbai/

Result: only city=Chennai/ is scanned. Remaining partitions are skipped.

This optimization is called Partition Pruning.

Advantages

- ✓ Faster Query Performance
- ✓ Less Disk I/O
- ✓ Less Network Traffic
- ✓ Better Scalability
- ✓ Reduced Execution Time
- ✓ Execution Engine reads only required partitions

Internal Working

```
SELECT *  
FROM sales  
WHERE city='Chennai';
```

```
User  
↓  
HiveServer2  
↓  
Driver  
↓  
Compiler  
↓  
Hive Metastore  
↓  
Checks Partition Metadata  
↓  
Finds city=Chennai  
↓  
HDFS Location  
↓  
/user/hive/warehouse/sales/city=Chennai/  
↓  
Execution Engine  
↓  
Read only Chennai files  
↓  
Output
```

Notice: Hive never scans Bangalore, Delhi or Mumbai folders. This is called Partition Pruning.

How Partition Information is Stored?

```
Table: sales  
↓  
Partition: city=Chennai  
↓  
Location:  
/user/hive/warehouse/sales/city=Chennai
```

Similarly for city=Bangalore → /user/hive/warehouse/sales/city=Bangalore

When user queries Chennai, Hive directly gets that location and asks HDFS only for that directory.

Physical Storage in HDFS

```
CREATE TABLE sales  
(  
  id INT,  
  customer STRING,  
  amount DOUBLE  
)  
PARTITIONED BY
```

```
(  
  city STRING  
);
```

Initially: /user/hive/warehouse/sales/

After inserting Chennai:

```
sales/  
city=Chennai/  
000000_0
```

After inserting Bangalore:

```
sales/  
city=Chennai/  
000000_0  
city=Bangalore/  
000000_0
```

After another insert into Chennai:

```
sales/  
city=Chennai/  
000000_0  
000001_0  
city=Bangalore/  
000000_0
```

Important Interview Point

Hive generally does not modify existing data files. Whenever new data is inserted, Hive usually creates new data files.

This is because HDFS follows a Write Once, Read Many model.

Static Partitioning

Definition

In Static Partitioning, the user explicitly specifies the partition value during data loading or insertion. Hive does not determine the partition — the user decides where the data should go.

Create Staging Table (Non-partitioned)

```
CREATE TABLE sales_stage  
(  
  id INT,  
  customer STRING,  
  amount DOUBLE,  
  city STRING  
);
```

Load Data

Local:

```
LOAD DATA LOCAL INPATH '/home/sales.csv'  
INTO TABLE sales_stage;
```

or HDFS:

```
LOAD DATA INPATH '/input/sales.csv'  
INTO TABLE sales_stage;
```

Create Partition Table

```
CREATE TABLE sales  
(  
  id INT,  
  customer STRING,  
  amount DOUBLE  
)  
PARTITIONED BY  
(  
  city STRING  
);
```

Notice: partition column city is not included in the normal columns because Hive stores it as a partition directory.

Insert Chennai

```
INSERT INTO TABLE sales  
PARTITION(city='Chennai')  
SELECT
```

```
id,  
customer,  
amount  
FROM sales_stage  
WHERE city='Chennai';
```

Insert Bangalore

```
INSERT INTO TABLE sales  
PARTITION(city='Bangalore')  
SELECT  
id,  
customer,  
amount  
FROM sales_stage  
WHERE city='Bangalore';
```

Every partition value must be specified manually.

Static Partition HDFS Structure

```
sales/  
city=Chennai/  
000000_0  
city=Bangalore/  
000000_0  
city=Delhi/  
000000_0
```

Static Partition Features

- ✓ User specifies partition value
- ✓ Supports LOAD DATA
- ✓ Supports INSERT INTO
- ✓ Supports INSERT OVERWRITE
- ✓ Good when partition values are known
- ✓ Faster because Hive doesn't determine partitions

Dynamic Partitioning

Definition

In Dynamic Partitioning, Hive automatically determines the partition value by reading each row. The user only specifies the partition column — Hive creates partition directories automatically.

Why LOAD DATA Doesn't Work?

```
LOAD DATA INPATH '/sales.csv'
```

Hive simply copies or moves the file — it never reads individual records. Since Hive doesn't inspect each row, it cannot determine city=?, state=?, year=?.

Therefore, Dynamic Partition requires INSERT, because INSERT reads every row.

Configuration Required

```
SET hive.exec.dynamic.partition=true;  
SET hive.exec.dynamic.partition.mode=nonstrict;
```

Without these, Hive throws an error.

Create Partition Table

```
CREATE TABLE sales_dynamic  
(  
id INT,  
customer STRING,  
amount DOUBLE  
)  
PARTITIONED BY  
(  
city STRING  
);
```

Insert Data

```
INSERT INTO TABLE sales_dynamic  
PARTITION(city)  
SELECT  
id,  
customer,  
amount,  
city
```

FROM sales_stage;

Notice we write PARTITION(city) NOT PARTITION(city='Chennai'). Hive automatically detects Chennai, Bangalore, Delhi, Mumbai and creates directories.

HDFS Structure

```
sales_dynamic/  
city=Chennai/  
000000_0  
city=Bangalore/  
000000_0  
city=Delhi/  
000000_0  
city=Mumbai/  
000000_0
```

Automatically created.

Dynamic Partition Internal Working

For every record, Hive checks:

Row → City → Chennai → Write inside city=Chennai

Next row:

Row → City → Bangalore → Write inside city=Bangalore

Hive repeats this for every row.

Static vs Dynamic Partition

Feature	Static	Dynamic
Partition Value	User specifies	Hive determines
LOAD DATA	✓ Supported	✗ Not Supported
INSERT INTO	✓ Supported	✓ Supported
INSERT OVERWRITE	✓ Supported	✓ Supported
Hive Reads Records?	Only during INSERT	Yes, every row
Partition Creation	Manual	Automatic
Performance	Slightly faster	Slightly slower during loading
Best Use	Known partition values	Unknown or many partition values

Multiple Partition Columns (Multi-level Partitioning)

Hive supports multiple partition columns.

```
CREATE TABLE sales  
(  
  id INT,  
  customer STRING,  
  amount DOUBLE  
)  
PARTITIONED BY  
(  
  country STRING,  
  state STRING  
);
```

HDFS Structure

```
sales/  
country=India/  
state=TamilNadu/  
000000_0  
state=Karnataka/  
000000_0  
country=USA/  
state=Texas/  
000000_0  
state=California/  
000000_0
```

Hive first partitions by country, and inside that by state.

Verify Partitions

```
SHOW PARTITIONS sales;
```

Example Output:

city=Bangalore
city=Chennai
city=Delhi
city=Mumbai

Describe Table

DESCRIBE FORMATTED sales;

Shows HDFS Location, Partition Information, Table Type, Owner, Storage Format.

See HDFS Directory

hdfs dfs -ls /user/hive/warehouse/sales

Output:

city=Bangalore
city=Chennai
city=Delhi

Check inside Chennai:

hdfs dfs -ls /user/hive/warehouse/sales/city=Chennai

Output:

000000_0
000001_0
000002_0

Best Practices (Interview)

Partition only on columns frequently used in WHERE clauses.

WHERE year=2026
WHERE month=07
WHERE city='Chennai'

Do NOT partition on very high-cardinality columns like Employee ID, Customer ID, Transaction ID.

This creates millions of tiny partitions, increasing metastore overhead and hurting performance.

Frequently Asked Interview Questions

Q: What is Partitioning?

A: Dividing a table into multiple directories based on partition column values so Hive scans only relevant data.

Q: What is Partition Pruning?

A: Hive skips unnecessary partition directories and scans only the matching partitions.

Q: Why is Partitioning Faster?

A: Because fewer files are read, reducing disk I/O and network traffic.

Q: Why doesn't Dynamic Partition support LOAD DATA?

A: LOAD DATA only moves/copies files. It does not inspect individual rows, so Hive cannot determine partition values.

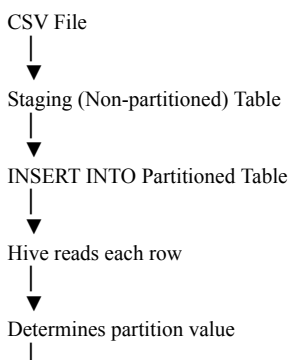
Q: Why does Dynamic Partition require INSERT?

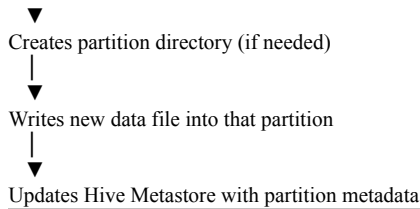
A: INSERT reads every row, allowing Hive to evaluate the partition column and place each row into the correct partition directory.

Q: Why does Hive create new files after every INSERT?

A: Hive follows HDFS's write-once, read-many design. Instead of modifying existing files, it appends new data by creating new files. Over time this can lead to the small files problem, which is mitigated using compaction (for ACID tables) or periodic file consolidation.

Complete Flow (Interview Summary)





★ Interview Tip (Very Important)

A common misconception is that partitioning moves rows inside a single file. It does NOT.

Hive partitioning works by creating separate HDFS directories, and each directory contains one or more data files.

```
/user/hive/warehouse/sales/  
├── city=Chennai/  
│   ├── 000000_0  
│   └── 000001_0  
├── city=Bangalore/  
│   └── 000000_0  
└── city=Delhi/  
    └── 000000_0
```

When you execute `SELECT * FROM sales WHERE city='Chennai';`, Hive consults the metastore, identifies the `city=Chennai` partition, and instructs the execution engine to read only the files inside that directory. This directory-based organization is the key reason partitioning provides significant performance improvements.

C. HIVE BUCKETING

Corrections

✗ 1. "Bucket automatically decides using Hash Partition"

The term Hash Partition is not normally used in Hive.

Correct interview answer: Hive uses a hashing function on the bucket column to determine the bucket number.

`Bucket Number = hash(bucket_column) % Number_of_Buckets`

This is the formula interviewers expect.

✗ 2. "Bucket is used only when values are unique"

Not true. Bucketing works with any column.

However, it is most beneficial on columns having High cardinality and Many distinct values.

Examples: ✓ Employee ID, ✓ Customer ID, ✓ Phone Number, ✓ Aadhaar Number, ✓ Transaction ID.

It also works on city, department, gender — but it won't be very useful.

✗ 3. "When duplicate values use partition"

This is slightly wrong. Correct statement: Partitioning works best on Low Cardinality Columns and Frequently filtered columns.

Examples: ✓ Year, ✓ Month, ✓ Date, ✓ Country, ✓ State, ✓ City. Not simply because there are duplicates.

✗ 4. Bucket Calculation (2300 MB / 128)

This is probably the biggest misconception. Hive does NOT calculate bucket count from HDFS block size.

Bucket count is not determined by Hive automatically — the developer decides it.

Then why do people teach `Table Size / Block Size ≈ Buckets`? It is only a rule of thumb, not an official Hive rule. It helps choose a reasonable bucket count.

✗ 5. Bucket count must be power of 2

Hive never requires that. You can create:

```
CLUSTERED BY(id)  
INTO 7 BUCKETS;
```

Perfectly valid. Production engineers often choose 8, 16, 32, 64, 128 because of balanced distribution, future scalability, and parallelism — not because Hive requires powers of two.

What is Bucketing?

Bucketing is a performance optimization technique in Hive that divides data into a fixed number of files (called buckets) based on the hash value of a column.

Unlike partitioning, Hive automatically decides which bucket each row belongs to.

Definition (Interview): Bucketing is the process of distributing table data into a fixed number of files using a hashing function on one or more bucket columns.

Why Bucketing?

Partitioning works well only for Low Cardinality columns: Year, Month, City, Country.

What if the column is Customer_ID, Phone_Number, Employee_ID, Transaction_ID? There may be millions of distinct values. Creating millions of partitions is impossible.

Therefore we use Bucketing.

Partition vs Bucket

Partition: City → Chennai, Bangalore, Delhi (Three directories)

Bucket: Customer_ID → Bucket 0, Bucket 1, Bucket 2, Bucket 3 (Four files)

Internal Working

CLUSTERED BY(customer_id)
INTO 4 BUCKETS;

Hive calculates:

Bucket Number = $\text{hash}(\text{customer_id}) \% 4$

Every record is assigned one bucket.

Mathematical Example (Numbers)

Suppose Buckets = 4

Customer ID 101 → $\text{hash}(101) = 325 \rightarrow 325 \% 4 = 1 \rightarrow \text{Bucket_00001}$

Customer ID 104 → $\text{hash}(104) = 500 \rightarrow 500 \% 4 = 0 \rightarrow \text{Bucket_00000}$

Customer ID 111 → $\text{hash}(111) = 900 \rightarrow 900 \% 4 = 0 \rightarrow \text{Bucket_00000}$

Mathematical Example (Strings)

Suppose Name = "Ram". Hive first converts "Ram" into an integer hash.

$\text{hash}(\text{Ram}) = 78965432$

Buckets = 8

$78965432 \% 8 = 0 \rightarrow \text{Bucket0}$

Suppose "John":

$\text{hash}(\text{John}) = 65478931$

$65478931 \% 8 = 3 \rightarrow \text{Bucket3}$

Important: You never calculate string hashes manually. Hive uses Java's internal hashing function. The interview expectation is simply that Hive converts the string into a hash value and applies the modulo operation to determine the bucket.

Physical Storage

CLUSTERED BY(customer_id)
INTO 4 BUCKETS;

sales/
000000_0
000001_0
000002_0
000003_0

Each file is one bucket.

How Hive Decides?

Suppose ID = 101, 102, 103, 104, 105, 106. After hashing:

Bucket0: 101, 105

Bucket1: 102

Bucket2: 103, 106

Bucket3: 104

Users never decide. Hive automatically assigns buckets.

Bucketing Syntax

```
CREATE TABLE customer
(
  id INT,
  name STRING,
  salary DOUBLE
)
CLUSTERED BY(id)
INTO 4 BUCKETS;
```

Enable Bucketing

```
SET hive.enforce.bucketing=true;
```

This forces Hive to write data according to bucket definitions.

Insert Data

```
INSERT INTO TABLE customer
SELECT *
FROM customer_stage;
```

Hive automatically places rows into the correct bucket.

Partition + Bucket

Most production tables use both.

```
CREATE TABLE sales
(
  id INT,
  customer STRING,
  amount DOUBLE
)
PARTITIONED BY
(
  year INT
)
CLUSTERED BY(id)
SORTED BY(id)
INTO 8 BUCKETS;
```

```
sales/
year=2025/
000000_0
000001_0
000002_0
...
year=2026/
000000_0
000001_0
000002_0
```

First Partition. Inside each partition, Buckets.

SORTED BY

```
CLUSTERED BY(id)
SORTED BY(id ASC)
INTO 8 BUCKETS;
```

Important: SORTED BY works only inside each bucket. It does NOT sort the entire table.

Bucket0: 11, 18, 45, 60

Bucket1: 7, 19, 80

Each bucket individually sorted.

Why Bucketing?

1. Faster Joins

Suppose Employee has 8 Buckets and Department has 8 Buckets, both bucketed on employee_id.

Hive can join Bucket0 with Bucket0 instead of scanning the complete table.

This is called a Bucket Map Join (or bucketed join when bucket compatibility exists). Huge performance improvement.

2. Efficient Sampling

Instead of reading 100 GB, read only Bucket2.

```
SELECT *
FROM customer
TABLESAMPLE(BUCKET 2 OUT OF 8 ON id);
```

Hive scans only Bucket2. Useful for Testing, ML, Analytics, Approximate Queries.

Bucket Count

Hive never calculates bucket count. Developer decides.

General guideline:

Bucket Count ≈ Table Size / HDFS Block Size

Example — Table = 2300 MB, Default Block = 128 MB

2300 / 128 ≈ 18

Choose 32 instead of 18 because of better parallelism, future growth, and easier scaling. This is a best practice, not a Hive rule.

Can Bucket Count be Increased?

No.

Bucket count cannot be changed using ALTER TABLE. You need a new table.

Old:

```
CLUSTERED BY(id)
INTO 8 BUCKETS;
```

Need 32 buckets — create:

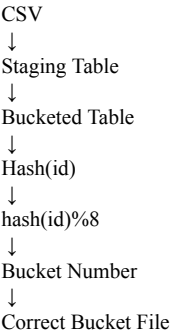
```
CREATE TABLE customer_new
...
CLUSTERED BY(id)
INTO 32 BUCKETS;
```

Move:

```
INSERT OVERWRITE TABLE customer_new
SELECT *
FROM customer;
```

Drop old table. Rename new table if required.

Bucketing Flow



Partition vs Bucketing

Feature	Partition	Bucketing
Data Division	Directories	Files
Based On	Partition Column Value	Hash Function
User Decides Location	Yes	No
Number	Variable	Fixed
Best For	Low Cardinality	High Cardinality
Query Optimization	Partition Pruning	Faster Joins & Sampling
Storage	Separate Folder	Separate File

Best Practices

Partition on:

- Year
- Month
- Country

- State
- City

Bucket on:

- Employee_ID
- Customer_ID
- Phone_Number
- Aadhaar_Number
- Transaction_ID

Use Partition + Bucket together for very large tables.

Interview Questions

Q: What is Bucketing?

A: Bucketing distributes data into a fixed number of files using a hashing function on the bucket column.

Q: Formula?

A: $\text{Bucket Number} = \text{hash}(\text{column}) \% \text{Number of Buckets}$

Q: Difference between Partition and Bucket?

A: Partition creates directories based on column values, while bucketing creates a fixed number of files using hash values.

Q: Why is Bucketing useful?

A: Faster joins, efficient sampling, better parallelism, balanced data distribution.

Q: Can bucket count be changed?

A: No. Create a new bucketed table and reload the data using `INSERT OVERWRITE TABLE new_table SELECT * FROM old_table;`

Q: Can we use Partition and Bucketing together?

A: Yes. This is common in production, e.g. `PARTITIONED BY(year) CLUSTERED BY(customer_id) SORTED BY(customer_id) INTO 16 BUCKETS`. Hive first creates partition directories (e.g. `year=2026`), and within each partition it creates the configured bucket files. This combination provides both partition pruning and efficient bucket-based processing for joins and sampling.

D. HIVE ACID PROPERTIES, ORC, PARQUET & SCRIPTS

Corrections

✗ 1. "Hive is now used for both OLTP and OLAP"

Not exactly. Hive is designed for OLAP.

Although ACID introduced INSERT, UPDATE, DELETE and MERGE, Hive is still not an OLTP database like MySQL or PostgreSQL.

Interview answer: Hive is primarily an OLAP (Online Analytical Processing) system. ACID transactions add support for data modifications, but Hive is not intended to replace OLTP databases because transaction latency is much higher.

✗ 2. "ACID table should be bucketed"

Historically (Hive 0.14–2.x), this was true. Modern Hive (Hive 3.x+) relaxed some restrictions.

However, for Hadoop interview preparation (Cloudera/CDH/HDP environments), remember: ACID tables are generally created as managed tables, stored in ORC format, and bucketed.

This is what most interviewers expect.

✗ 3. "Insert overwrite allowed"

Not always. For full ACID transactional tables, INSERT OVERWRITE has restrictions because overwriting data can violate transactional consistency.

Safer interview answer: ✓ INSERT, ✓ UPDATE, ✓ DELETE, ✓ MERGE are ACID operations.

Mention INSERT OVERWRITE only when discussing non-transactional tables unless the interviewer asks about specific Hive versions.

✗ 4. "ORC is 75% smaller"

No fixed percentage.

Correct statement: ORC typically compresses data much better than plain text or CSV, often reducing storage by 50–75% or more, depending on the data and compression settings. Never say exactly 75%.

Why ACID was Introduced?

Initially, Hive supported only Read and Insert. No Update, Delete, or Merge.

This was sufficient for analytics but not for modifying existing records. To support transactional operations, Hive introduced ACID tables.

What is ACID?

A → Atomicity
C → Consistency
I → Isolation
D → Durability

Atomicity

Either everything happens or nothing happens.

```
UPDATE employee  
SET salary=50000  
WHERE id=10;
```

Suppose power fails halfway. Atomicity guarantees the entire update succeeds OR the entire update is rolled back. No partial updates.

Consistency

Database always remains valid. Suppose salary cannot be negative:

```
UPDATE employee  
SET salary=-100;
```

Transaction fails. Database remains consistent.

Isolation

Suppose User1 updates salary. User2 should not see half-completed data. Every transaction remains isolated.

Durability

Once transaction commits, even if server crashes, changes remain permanent.

Requirements for Hive ACID Tables

For traditional Hive interview environments:

1. Managed Table

```
CREATE TABLE employee  
...
```

Not CREATE EXTERNAL TABLE. Reason: Hive must control the data lifecycle.

2. ORC Storage Format

```
STORED AS ORC
```

Reason: ACID implementation is built around ORC's efficient storage structures.

3. Bucketing

```
CLUSTERED BY(id)  
INTO 8 BUCKETS;
```

Reason: Buckets help organize transactional writes and improve update/delete processing in classic Hive implementations.

4. Transaction Property

```
TBLPROPERTIES  
(  
'transactional'='true'  
);
```

This enables transactional behavior.

Complete ACID Table Example

```
CREATE TABLE employee  
(  
  id INT,  
  name STRING,  
  salary DOUBLE  
)  
CLUSTERED BY(id)  
INTO 8 BUCKETS  
STORED AS ORC
```

```
TBLPROPERTIES
(
'transactional'='true'
);
```

Operations Supported

```
INSERT
UPDATE
DELETE
MERGE
```

These operations are logged transactionally.

Internal Working

```
UPDATE employee
SET salary=70000
WHERE id=10;
```

Hive does not rewrite the entire ORC file immediately. Instead it writes delta files.

```
employee/
base_000001/
delta_000002_000002/
delete_delta_000003/
```

Later, Compaction merges these files into a new optimized base file. This avoids rewriting huge files for every small update.

Why Hive is Still OLAP

Even with ACID, Hive is optimized for Large scans, Aggregations, Reporting, Batch analytics.

It is NOT optimized for thousands of small transactions per second like MySQL.

ORC File Format

What is ORC?

ORC stands for Optimized Row Columnar. It is a highly optimized columnar storage format for Hive.

Why ORC?

Suppose a table with ID, Name, Salary, Department columns. CSV stores rows:

```
1,Ram,50000,IT
2,John,60000,HR
```

Entire row must be read.

ORC stores each column separately:

```
ID:      1, 2, 3, 4
Name:    Ram, John, David
Salary:  50000, 60000, 70000
Department: IT, HR, Sales
```

Why Faster?

```
SELECT salary
FROM employee;
```

CSV reads ID, Name, Salary, Department. ORC reads only Salary. Huge reduction in I/O.

Compression

CSV: 100 GB. ORC may become 25–50 GB depending on Data, Compression algorithm, Repeated values.

Less storage, less disk I/O, less network traffic, faster execution.

ORC Advantages

- ✓ Columnar Storage
- ✓ Better Compression
- ✓ Predicate Pushdown
- ✓ Statistics
- ✓ Faster Aggregation
- ✓ Faster Filtering
- ✓ Better Performance
- ✓ ACID Support

Parquet File Format

What is Parquet?

Apache Parquet is also a columnar storage format. Unlike ORC, Parquet is designed for a wider ecosystem.

ORC vs Parquet

Feature	ORC	Parquet
Developed For	Hive	Hadoop Ecosystem
Best With	Hive	Spark, Hadoop, Cloud
Compression	Excellent	Excellent
ACID Support	Yes (Hive)	No native Hive ACID
Predicate Pushdown	Yes	Yes
Columnar	Yes	Yes

When to Use ORC?

Choose ORC if mainly using Hive, need ACID support, heavy analytical queries, large aggregations.

When to Use Parquet?

Choose Parquet if Spark, Databricks, Snowflake, BigQuery, AWS Glue, Azure Synapse, or multiple engines will access the data.

It has become the de facto standard in modern data lakes.

Why Columnar Storage is Faster?

```
SELECT AVG(salary)
FROM employee;
```

Need only Salary. ORC reads only Salary column. CSV reads entire table.

Therefore columnar storage greatly improves Aggregation, Filtering, Compression.

Hive Script (.hql / .sql)

Instead of typing commands one by one, we can store them in a script file.

```
CREATE DATABASE company;
USE company;
CREATE TABLE employee
(
  id INT,
  name STRING
);
LOAD DATA LOCAL INPATH '/home/employee.csv'
INTO TABLE employee;
SELECT * FROM employee;
```

Save as employee.hql or employee.sql

Execute Script

Basic:

```
hive -f employee.hql
```

-f means: Execute commands from a file.

Verbose Mode

```
hive -v -f employee.hql
```

-v (Verbose): Displays each HiveQL statement before executing it. Useful for debugging and understanding execution.

Silent Mode

```
hive -S -f employee.hql
```

-S (Silent — uppercase S): Suppresses most log messages and prints mainly the query results. Useful when redirecting output to another file or script.

Correction: The option is -S (uppercase), not -s.

Redirect Output

```
hive -S -f employee.hql > output.txt
```

Stores query result in output.txt

Useful Hive Commands

Execute single query:

```
hive -e "SELECT * FROM employee;"
```

Execute script:

```
hive -f employee.hql
```

Verbose execution:

```
hive -v -f employee.hql
```

Silent execution:

```
hive -S -f employee.hql
```

Interview Questions

Q: Why is Hive still considered OLAP?

A: Because it is optimized for large-scale analytical queries, not high-frequency transactional workloads.

Q: What are the requirements for an ACID table?

A: For traditional Hive environments: managed table, ORC storage, bucketed, transactional=true.

Q: Why ORC instead of CSV?

A: Columnar storage, better compression, faster scans, predicate pushdown, ACID support.

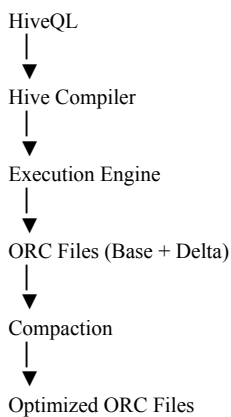
Q: Difference between ORC and Parquet?

A: ORC is optimized for Hive and supports Hive ACID, while Parquet is a general-purpose columnar format widely used across Spark, cloud data lakes, and many analytics engines.

Q: How does Hive update a row?

A: Hive does not modify the original ORC file immediately. It writes transactional delta and delete_delta files, and later compaction merges them into a new optimized base file.

Complete Flow (Interview Summary)



E. CREATE TABLE CLAUSES — ROW FORMAT, SERDE, STORED AS, LOCATION

General Hive CREATE TABLE Syntax

```
CREATE [EXTERNAL] TABLE table_name
(
    col1 INT,
    col2 STRING,
    col3 DOUBLE
)
PARTITIONED BY(...)
CLUSTERED BY(...)
SORTED BY(...)
INTO n BUCKETS
ROW FORMAT ...
STORED AS ...
LOCATION ...
TBLPROPERTIES(...);
```

Every line controls how Hive stores and reads the data.

1. ROW FORMAT

This tells Hive: "How should I interpret the incoming data?"

Suppose your file looks like:

```
1,Ram,50000
2,John,60000
3,David,70000
```

How does Hive know where one column ends and where the next begins? It doesn't — you tell Hive.

ROW FORMAT DELIMITED

ROW FORMAT DELIMITED

Means: the file is a plain text file. Now tell Hive how columns are separated.

FIELDS TERMINATED BY

FIELDS TERMINATED BY ','

Input file:

```
1,Ram,50000
```

Hive understands: Column1 → 1, Column2 → Ram, Column3 → 50000

Suppose file "1|Ram|50000" — then FIELDS TERMINATED BY '|'. Suppose "1#Ram#50000" — then FIELDS TERMINATED BY '#'.

LINES TERMINATED BY

Normally \n means next row.

```
1,Ram,50000\n
2,John,60000\n
3,David,70000\n
```

Hive reads: every \n → New Record

Complete Example

```
CREATE TABLE employee
(
  id INT,
  name STRING,
  salary DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n';
```

When is ROW FORMAT DELIMITED used?

Mainly CSV, TSV, Pipe-separated files, Flat files.

What about ORC?

When we write STORED AS ORC, Hive doesn't need FIELDS TERMINATED BY ',' because ORC isn't a text file — it is a binary structured file.

CSV (human readable):

```
1,Ram,50000
```

ORC (binary):

```
010110101001011000101001...
```

Hive already knows ORC's internal structure. So no delimiter is required.

2. ROW FORMAT SERDE

```
ROW FORMAT SERDE
'org.apache.hadoop.hive ql.io.orc.OrcSerde'
```

SERDE means Serializer + Deserializer.

What is Serialization?

Suppose Hive has Ram, 50000, IT. It converts it into ORC binary format. That process is Serialization.

What is Deserialization?

Suppose an ORC file contains binary data. Hive converts it back into Ram, 50000, IT. That is Deserialization.

So ROW FORMAT SERDE 'OrcSerde' means: Use the ORC Serializer/Deserializer.

3. STORED AS INPUTFORMAT

```
STORED AS INPUTFORMAT
'org.apache.hadoop.hive ql.io.orc.OrcInputFormat'
```

Meaning: when reading data, use ORC Reader.

```
SELECT *  
FROM employee;
```

Hive uses OrcInputFormat to read ORC files.

4. OUTPUTFORMAT

```
OUTPUTFORMAT  
'OrcOutputFormat'
```

Meaning: when writing (INSERT INTO employee), Hive uses OrcOutputFormat to write ORC files.

Simplified Flow

```
SELECT  
↓  
OrcInputFormat  
↓  
Read ORC  
↓  
Deserializer  
↓  
Hive Columns
```

```
INSERT  
↓  
Serializer  
↓  
OrcOutputFormat  
↓  
ORC File
```

5. STORED AS ORC

Instead of writing the full ROW FORMAT SERDE ... INPUTFORMAT ... OUTPUTFORMAT ... block, we simply write:

```
STORED AS ORC;
```

Hive automatically fills all these — equivalent to:

```
ROW FORMAT SERDE  
'OrcSerde'  
STORED AS INPUTFORMAT  
'OrcInputFormat'  
OUTPUTFORMAT  
'OrcOutputFormat'
```

Same for Parquet

```
STORED AS PARQUET;
```

Hive automatically uses Parquet Reader, Parquet Writer, Parquet SerDe.

6. LOCATION

```
LOCATION  
'hdfs://localhost:50000/user/hive/warehouse/internal_table'
```

Meaning: store data here. Instead of default /user/hive/warehouse/, you explicitly tell Hive where data should live.

```
CREATE EXTERNAL TABLE employee  
...  
LOCATION  
'/project/sales/';
```

Hive stores metadata pointing to /project/sales/ instead of warehouse.

Internal Table

Usually /user/hive/warehouse/employee/

External

May be /company/raw/employee/ or /data/project/employee/

Modern Syntax

Today, almost everyone writes:

```
CREATE TABLE employee
```

```
(
  id INT,
  name STRING,
  salary DOUBLE
)
STORED AS ORC;
```

instead of 100 lines of SERDE / INPUTFORMAT / OUTPUTFORMAT, because Hive automatically chooses them.

Common Storage Formats

CSV

```
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

TSV

```
FIELDS TERMINATED BY '\t'
```

Pipe File

```
FIELDS TERMINATED BY '|'
```

ORC

```
STORED AS ORC;
```

Parquet

```
STORED AS PARQUET;
```

Interview Questions

Q: What is SerDe?

A: SerDe (Serializer/Deserializer) converts Hive table data into the storage format while writing (serialization) and converts stored data back into rows and columns while reading (deserialization).

Q: Why doesn't ORC need FIELDS TERMINATED BY?

A: Because ORC is a structured binary file format. It stores schema and data internally, so Hive already knows how to interpret it.

Difference between ROW FORMAT DELIMITED and ROW FORMAT SERDE?

ROW FORMAT DELIMITED	ROW FORMAT SERDE
Used for text files (CSV, TSV, etc.)	Used for structured formats (ORC, Parquet, Avro, JSON with custom SerDes)
User specifies delimiters	SerDe handles encoding and decoding automatically

Q: Why do we specify LOCATION?

A: To tell Hive the exact HDFS directory where the table's data is stored or should be stored. This is especially common for external tables, where the data already exists in HDFS.

One Important Interview Tip

If you ever run:

```
SHOW CREATE TABLE employee;
```

Hive may display the expanded form with ROW FORMAT SERDE ..., STORED AS INPUTFORMAT ..., OUTPUTFORMAT ... even though you originally created the table using STORED AS ORC;

This is because STORED AS ORC is just a shorthand. Internally, Hive expands it into the full SerDe, InputFormat, and OutputFormat definitions. This is normal behavior and a common interview discussion point.

F. HIVE COMPACTION & DELTA FILES

What is Hive Compaction?

Definition (Interview): Hive Compaction is the process of merging multiple small delta files and base files into a new optimized ORC base file to improve query performance and reduce the number of files.

Why is Compaction Needed?

Remember, Hive follows the Write Once, Read Many principle of HDFS.

Suppose we have an ACID table. Initially:

employee/
base_000001/

Contains:

ID	Name	Salary
1	Ram	50000
2	John	60000
3	David	70000

Now execute:

```
UPDATE employee  
SET salary=55000  
WHERE id=1;
```

Hive does not modify base_000001. Instead it creates:

employee/
base_000001/
delta_000002/

Now execute:

```
DELETE FROM employee  
WHERE id=2;
```

Hive creates:

employee/
base_000001/
delta_000002/
delete_delta_000003/

Now execute:

```
INSERT INTO employee  
VALUES(4,'Sam',65000);
```

Hive creates:

employee/
base_000001/
delta_000002/
delete_delta_000003/
delta_000004/

Now imagine 1000 updates. HDFS becomes:

employee/
base_000001/
delta_000002/
delta_000003/
delta_000004/
...
delta_001000/

Thousands of files.

Problem Without Compaction

Suppose user executes:

```
SELECT *  
FROM employee;
```

Hive now has to read base_000001 + delta_000002 + delta_000003 + ... + delta_001000, then merge them logically before showing results.

- ✗ More file openings
- ✗ More metadata lookups
- ✗ More disk seeks
- ✗ More memory usage
- ✗ Slower queries
- ✗ Small File Problem

Solution — Hive Runs Compaction

Compaction merges everything.

Before:

employee/
base_000001/
delta_000002/
delta_000003/
delta_000004/

After:

employee/
base_000005/

Only one optimized file. Old delta files are deleted after cleanup.

Types of Compaction

1. Minor Compaction

Merges only delta + delta + delta

Before:

delta_2
delta_3
delta_4

After:

delta_5

Base file remains unchanged.

2. Major Compaction

Merges Base + All Delta Files

Before:

base_1
delta_2
delta_3
delta_4

After:

base_5

Everything becomes one new base file. This gives the best read performance.

Compaction Flow

INSERT
↓
delta file
↓
UPDATE
↓
delta file
↓
DELETE
↓
delete_delta
↓
Many small files
↓
Compaction
↓
One optimized ORC base file

How Often Does Hive Compact?

There is no fixed schedule. Hive can perform compaction:

1. Automatically (Most Common)

Hive has a Compactor service running in the background. It checks transactional tables periodically. When the number or size of delta files crosses configured thresholds, it schedules compaction automatically.

2. Manually

Administrator can run:

ALTER TABLE employee COMPACT 'MINOR';
or
ALTER TABLE employee COMPACT 'MAJOR';

Does Compaction Run Every Day?

No. It depends on Number of delta files, Size of delta files, Hive configuration, Workload.

A table updated thousands of times per hour may compact frequently. A table updated once a week may hardly ever compact.

Important Hive Compaction Configuration

hive.compactor.initiator.on=true

Starts the background compaction scheduler.

hive.compactor.worker.threads=1

Number of worker threads performing compaction. Increase in production if many ACID tables exist.

Cleaner Process

Compaction creates base_000005. Old files (base_000001, delta_2, delta_3, delta_4) are not immediately deleted.

Why? Because some users may still be reading them.

A background Cleaner waits until no active transaction needs those files, then safely deletes them.

Advantages of Compaction

- ✓ Faster SELECT queries
- ✓ Fewer HDFS files
- ✓ Reduces Small File Problem
- ✓ Lower metadata overhead
- ✓ Better ORC compression
- ✓ Faster scans
- ✓ Better join performance

What Happens If We Don't Compact?

Suppose one table receives 100 updates every hour. After one month:

base_1
delta_2
delta_3
delta_4
...
delta_72000

Now every query must read 72,001 files. Performance becomes very poor.

NameNode metadata also increases because every HDFS file consumes metadata memory.

Eventually: Queries become slower. More memory is used by Hive and HDFS. More time is spent opening and merging files.

Real Company Practice

In production:

- Most teams enable automatic compaction.
- Administrators monitor compaction jobs.
- Large, frequently updated ACID tables are periodically checked to ensure compaction keeps up with incoming updates.
- Tables that are mostly append-only may require little or no compaction.

Interview Questions

Q: Why is compaction required?

A: Because Hive ACID creates many delta and delete_delta files during INSERT, UPDATE, and DELETE operations. Compaction merges these into optimized files, improving read performance.

Q: What are the types of compaction?

A: Minor Compaction merges only delta files. Major Compaction merges the base file and all delta files into a new base file.

Q: Is compaction mandatory?

A: No. Hive can still read the data without compaction. However, performance degrades over time as the number of delta files increases.

Q: Who triggers compaction?

A: Either Hive's automatic background Compactor service, or an administrator using ALTER TABLE employee COMPACT 'MAJOR'; or ALTER TABLE employee COMPACT 'MINOR';

Complete ACID Write Flow

INSERT / UPDATE / DELETE



Base File + Delta Files + Delete_Delta Files



Hive Compactor



Minor or Major Compaction



New Optimized ORC Base File



Cleaner Deletes Old Files



Fast Query Performance

Interview Tip: A common question is, "If Hive supports UPDATE, why isn't it an OLTP database?" The answer is that every update creates delta files instead of modifying rows in place. Over time, these delta files require compaction, making updates much heavier than in databases like MySQL or PostgreSQL. That's why Hive remains optimized for analytical (OLAP) workloads rather than high-frequency transactional (OLTP) workloads.

Delta File — Detailed Walkthrough

Now execute:

```
UPDATE employee  
SET salary=55000  
WHERE id=1;
```

Hive does not modify base_000001. Instead it creates:

```
employee/  
base_000001/  
delta_000002/
```

The delta file contains only the changed rows.

Contents:

Operation	ID	Salary
UPDATE	1	55000

Notice: only one row — not the whole table.

Another UPDATE

```
UPDATE employee  
SET salary=65000  
WHERE id=2;
```

Hive creates:

```
employee/  
base_000001/  
delta_000002/  
delta_000003/
```

Contents of delta_000003:

Operation	ID	Salary
UPDATE	2	65000

DELETE

Suppose:

```
DELETE FROM employee  
WHERE id=3;
```

Hive creates:

```
delete_delta_000004/
```

Contents:

Operation	ID
DELETE	3

It doesn't physically remove the row from the base file immediately.

INSERT

```
INSERT INTO employee  
VALUES(4,'Sam',80000);
```

Hive creates:

delta_000005/

Contents:

Operation	ID	Name	Salary
INSERT	4	Sam	80000

What Does Major Compaction Do?

Before:

employee/
base_000001/
delta_000002/
delta_000003/
delete_delta_000004/
delta_000005/

Hive reads five different files.

After:

```
ALTER TABLE employee COMPACT 'MAJOR';
```

Hive creates:

employee/
base_000006/

Contents:

ID	Name	Salary
1	Ram	55000
2	John	65000
4	Sam	80000

Everything is now inside one new base file.

Old delta files are later removed by the Cleaner process.

LECTURE 6 — APACHE SQOOP

1. What is Sqoop?

Sqoop (SQL-to-Hadoop) is an Apache tool used to transfer bulk data between relational databases (RDBMS) and the Hadoop ecosystem.

It is mainly used to:

- Import data from RDBMS → HDFS/Hive/HBase
- Export data from HDFS/Hive → RDBMS

Sqoop is designed for bulk data transfer, not real-time streaming.

2. Why Sqoop?

Suppose a company stores customer data in MySQL:

Customer Database (MySQL)

|

10 Million Records

|
Need analysis in Hive/Spark

Without Sqoop:

- ✗ Write Java/Python code
- ✗ Connect via JDBC
- ✗ Read records
- ✗ Write to HDFS
- ✗ Handle parallelism manually

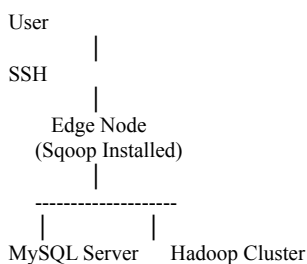
With Sqoop:

MySQL
|
Sqoop
|
HDFS/Hive

One command imports millions of records.

3. Where is Sqoop Installed?

Sqoop is installed on an Edge Node (Gateway Node) where Hadoop client tools are available.



You run Sqoop commands from the Linux terminal of the Edge Node, not inside the MySQL prompt or Hive shell.

`sqoop import ...`

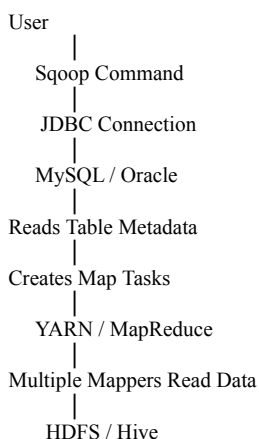
4. Why Run from the Edge Node?

The Edge Node has:

- Sqoop binaries
- Hadoop client
- HDFS client
- Hive client
- JDBC drivers
- Network access to the database

The Edge Node submits MapReduce/YARN jobs to the cluster.

5. Sqoop Architecture



6. Components

Sqoop Client

Accepts your command. Example:

`sqoop import`

JDBC Driver

Sqoop communicates with databases using JDBC. Different databases need different drivers:

- MySQL Connector/J
- Oracle JDBC
- PostgreSQL JDBC
- SQL Server JDBC

Without the driver, Sqoop cannot connect.

Database

Can be MySQL, Oracle, PostgreSQL, SQL Server, DB2.

MapReduce/YARN

Sqoop launches a MapReduce job (or YARN application) for parallel data transfer.

HDFS/Hive/HBase

Destination where imported data is stored.

7. How Sqoop Works (Import)

Suppose MySQL has an Employee Table with 1,000,000 rows. You run `sqoop import ...` — Sqoop:

- 1. Connects to MySQL.
- 2. Reads table metadata.
- 3. Finds split column (usually primary key).
- 4. Divides data into ranges.
- 5. Launches multiple mappers.
- 6. Each mapper reads one range.
- 7. Writes output to HDFS.
- 8. No Reducers are used.

8. Parallel Import

Example — Employee ID range 1 – 1,000,000, using 4 mappers:

Mapper	ID Range
Mapper 1	1 – 250000
Mapper 2	250001 – 500000
Mapper 3	500001 – 750000
Mapper 4	750001 – 1000000

All run simultaneously.

9. Why No Reducers?

Sqoop simply copies data. No aggregation, sorting, grouping, or joins.

Therefore: Only Mappers. Reducers = 0.

10. General Commands

Command	Purpose
sqoop version	Displays the installed Sqoop version
sqoop help	Shows all available Sqoop tools (import, export, eval, list-tables, list-databases, codegen, create-hive-table, merge, job, metastore)
sqoop import --help	Shows every option available for the import command

11. Database Information Commands

Command	Purpose
sqoop list-databases --connect jdbc:mysql://host:3306 --username u --password p	Lists all databases
sqoop list-tables --connect jdbc:mysql://host:3306/db --username u --password p	Lists all tables
sqoop eval --connect ... --query "SELECT * FROM employee"	Runs any SQL query (e.g. DESCRIBE employee)

12. Import Commands

Basic import syntax:

```
sqoop import \  
--connect jdbc:mysql://hostname:3306/database \  
--username username \  
--password password \  
--table table_name \  
--target-dir /path
```

Option	Purpose
--table employee (no --target-dir)	Imports entire table; default output goes into HDFS warehouse path
--columns id,name,salary	Imports only the selected columns
--where "salary>50000"	Imports only matching rows
-m 1	One mapper — used when no primary key, small table, or to avoid overloading the database
-m 4 or --num-mappers 4	Number of parallel mappers (default 4); more mappers = more parallelism, up to DB capacity
--split-by employee_id	Column used to divide work across mappers when no suitable primary key exists
--direct	Uses database-native utilities when supported (faster)
--delete-target-dir	Deletes the HDFS target directory before import — useful when rerunning
--append	Appends to existing files instead of overwriting
--warehouse-dir /warehouse	Stores imported tables under a warehouse directory
--bindir /home/hadoop/generated	Stores generated Java classes
--temporary-rootdir /tmp	Temporary root directory used during import

Free-form Query Import

```
sqoop import \  
--query "SELECT id,name FROM employee WHERE $CONDITIONS" \  
--split-by id \  
--target-dir /employee
```

\$CONDITIONS is mandatory so Sqoop can inject mapper-specific conditions.

13. Hive Import

```
sqoop import \  
--connect jdbc:mysql://localhost/company \  
--username root \  
--password root \  
--table employee \  
--hive-import \  
--create-hive-table \  
--hive-table employee
```

Option	Purpose
--hive-import	Loads imported data directly into Hive
--create-hive-table	Creates the Hive table (optional)
--hive-database salesdb	Specifies the target Hive database
--hive-table employee	Specifies the target Hive table name
--hive-overwrite	Overwrites existing Hive table data
--hive-drop-import-delims	Removes newline/tab characters that could break Hive's row format

14. HBase Import

Option	Purpose
--hbase-table employee	Target HBase table
--column-family info	Column family to store imported fields
--hbase-create-table	Creates the HBase table if it doesn't exist

15. Export Commands

Reverse operation:

HDFS

↓

Sqoop

↓

MySQL

```
sqoop export \
--connect jdbc:mysql://localhost/company \
--username root \
--password root \
--table employee \
--export-dir /employee
```

Option	Purpose
--update-key id	Updates existing rows matched by this key instead of inserting new ones
--update-mode allowinsert	Updates matching rows and inserts new ones
--update-mode updateonly	Only updates existing rows (default)
--input-fields-terminated-by ','	Field delimiter used to parse the HDFS input during export

16. Incremental Import

Only imports newly added or updated data.

Append Mode

Uses increasing ID.

Example: already imported ID 1-100. Database now has 101, 102, 103. Sqoop imports only 101-103.

```
--incremental append \
--check-column id \
--last-value 100
```

Last Modified Mode

Uses timestamp — checks modified timestamp.

```
--incremental lastmodified
```

17. File Formats

Option	Purpose
(default)	Text file
--as-sequencefile	Hadoop SequenceFile
--as-avrodatafile	Avro file
--as-parquetfile	Parquet (with compatible connectors/tools)

18. Delimiters

Option	Purpose
--fields-terminated-by ','	Field delimiter
--lines-terminated-by '\n'	Line delimiter
--optionally-enclosed-by '"'	Optional enclosing character for fields
--escaped-by '\\'	Escape character

19. Compression

```
--compress  
--compression-codec org.apache.hadoop.io.compress.SnappyCodec
```

20. Null Handling

```
--null-string '\\N'  
--null-non-string '\\N'
```

21. Security

Option	Purpose
--password-file /user/password.txt	Avoids exposing passwords in command history
-P	Prompts for the password interactively

22. Code Generation

Generate Java classes:

```
sqoop codegen \  
--connect jdbc:mysql://localhost/company \  
--username root \  
--password root \  
--table employee
```

23. Merge

```
sqoop merge \  
--new-data /new \  
--onto /old \  
--target-dir /merged \  
--merge-key id
```

Merges incremental datasets.

24. Job Commands

```
sqoop job --create job1 -- import \  
--connect jdbc:mysql://localhost/company \  
--table employee
```

Command	Purpose
sqoop job --exec job1	Executes a saved job
sqoop job --list	Lists all saved jobs
sqoop job --delete job1	Deletes a saved job
sqoop job --show job1	Shows job details

25. Metastore Commands

```
sqoop metastore
```

Starts the Sqoop Metastore.

```
--meta-connect jdbc:hsqldb:hsql://localhost/sqoop
```

Connects to the Metastore.

26. Create Hive Table (Standalone Tool)

```
sqoop create-hive-table \  
--connect jdbc:mysql://localhost/company \  
--table employee \  
--hive-table employee
```

Only creates the Hive table; it does not import data.

27. Common JDBC URLs

Database	JDBC URL
MySQL	<code>jdbc:mysql://localhost:3306/company</code>
Oracle	<code>jdbc:oracle:thin:@localhost:1521:xe</code>
PostgreSQL	<code>jdbc:postgresql://localhost:5432/company</code>
SQL Server	<code>jdbc:sqlserver://localhost:1433;databaseName=company</code>

28. Complete Import Examples

Example 1 — Compressed Import with Split

```
sqoop import \  
--connect jdbc:mysql://192.168.1.10:3306/company \  
--username root \  
--password root \  
--table employee \  
--target-dir /data/employee \  
--num-mappers 4 \  
--split-by id \  
--compress \  
--delete-target-dir
```

Connects to MySQL, imports the employee table using 4 parallel mappers split on id, compresses output, and deletes the target directory before writing new data.

Example 2 — Industry Import (Secure Password Prompt)

```
sqoop import \  
--connect jdbc:mysql://192.168.1.100:3306/company \  
--username admin \  
-P \  
--table employee \  
--target-dir /warehouse/employee \  
--num-mappers 4 \  
--split-by id \  
--fields-terminated-by ',' \  
--lines-terminated-by '\n' \  
--compress \  
--compression-codec org.apache.hadoop.io.compress.SnappyCodec \  
--delete-target-dir
```

29. Real Industry ETL Example

```
MySQL (Orders)  
|  
1 AM: Sqoop Import  
|  
HDFS  
|  
Hive Cleansing  
|  
Spark Transformations  
|  
Hive Data Warehouse  
|  
BI Dashboard
```

Oozie or Airflow schedules this pipeline daily.

30. Important Linux Commands (Run on the Edge Node)

```
sqoop version      # Check Sqoop version
sqoop help         # General help
sqoop import --help # Import command options
sqoop export --help # Export command options
hdfs dfs -ls /data # Verify imported files in HDFS
hdfs dfs -cat /data/employee/part-m-00000 # View imported data
```

Most Important Commands to Memorize (Interview Focus)

```
sqoop version
sqoop help
sqoop list-databases
sqoop list-tables
sqoop eval
sqoop import
sqoop export
sqoop job
sqoop codegen
sqoop merge
sqoop create-hive-table
sqoop metastore
```

These cover 95%+ of what is expected in Hadoop and Data Engineering interviews. The remaining options are specialized and are typically looked up from the documentation when needed.

Common Interview Questions

Q: Does Sqoop use MapReduce?

A: Yes. It launches a MapReduce (or YARN) job with only mapper tasks for data transfer.

Q: Why are there no reducers?

A: Because Sqoop only copies data. It does not aggregate, sort, or join records.

Q: Why is a split column required?

A: To divide data among multiple mappers for parallel import.

Q: What happens if a table has no primary key?

A: Use --split-by on another indexed numeric column, or run with -m 1 if parallel splitting isn't possible.

Q: Can Sqoop import directly into Hive?

A: Yes, using --hive-import.

Q: Is Sqoop suitable for real-time data ingestion?

A: No. It is intended for batch transfers. Technologies like Kafka are used for real-time streaming.

One-Line Revision

```
Sqoop      = Batch data transfer between RDBMS ↔ Hadoop
Runs on    = Edge/Gateway node Linux terminal
Connects via = JDBC driver
Execution  = MapReduce/YARN with mappers only
Import     = Database → HDFS/Hive/HBase
Export     = HDFS/Hive → Database
Parallelism = Controlled by -m / --num-mappers
Split column = Used to divide data across mappers
Incremental = append or lastmodified
Not for real-time = Use Kafka or CDC tools for streaming data
```

SPARK — LECTURE 1: INTRODUCTION

A. WHY APACHE SPARK & CORE CONCEPTS

Why Apache Spark?

As data started growing from GB → TB → PB, a single computer was no longer enough. There are two ways to increase computing power.

1. Vertical Scaling (Monolithic System)

Increase resources of the same machine.

One Powerful Machine

+-----+

```

| CPU ↑      |
| RAM ↑      |
| Storage ↑  |
+-----+

```

Example: 8 GB RAM → 64 GB RAM, 4 Cores → 32 Cores

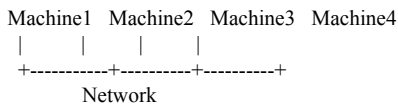
Advantages	Disadvantages
Easy to manage	Very expensive
No network communication	Hardware limit exists
	Single Point of Failure
	Limited scalability

If this server crashes, everything stops.

2. Horizontal Scaling (Distributed System)

Instead of upgrading one machine, we add multiple computers. Data gets divided across all machines.

Cluster



- ✓ Cheap
- ✓ Easily scalable
- ✓ High Availability
- ✓ Fault Tolerance
- ✓ Faster parallel processing

Why Distributed Computing?

Suppose you have 100 TB Data. One computer may take 15 Hours. Ten computers process different parts simultaneously:

Computer 1 -> 10 TB
 Computer 2 -> 10 TB
 Computer 3 -> 10 TB
 ...
 Computer10 -> 10 TB

Time reduces dramatically because work happens in parallel.

What is Apache Spark?

Definition (Interview): Apache Spark is an open-source distributed parallel data processing engine that processes massive datasets across a cluster of machines using in-memory computation for high-speed analytics.

Simple definition: Spark is a distributed computing engine used to process large amounts of data in parallel across multiple machines.

Who Developed Spark?

Developed at AMP Lab, University of California, Berkeley. Later donated to Apache Foundation.

Why was Spark Introduced?

Before Spark, people mainly used Hadoop MapReduce. MapReduce was reliable but slow. Spark solved this speed problem.

MapReduce vs Spark

Hadoop MapReduce

```

Read Data
↓
Map
↓
Write to Disk
↓
Read Again
↓
Reduce
↓
Write Again

```

Every stage writes intermediate results to disk. Disk I/O is slow.

Apache Spark

Read Data



Memory (RAM)



Transformations



Memory



Result

Intermediate results stay in RAM. Only final output is written to storage. Hence much faster.

Is Spark Always 100x Faster?

Not always.

Interview answer: Spark can be up to 100× faster in memory and around 10× faster on disk compared to Hadoop MapReduce, depending on the workload.

Main Features of Spark

1. In-Memory Computation

Stores intermediate data in RAM. Less disk access. Very fast.

2. Distributed Processing

Data is divided among multiple machines. Processing happens simultaneously.

3. Parallel Processing

Many CPU cores execute tasks together.

4. Lazy Evaluation

Spark doesn't execute transformations immediately. It waits until an Action is called. This allows Spark to optimize the execution plan.

5. Fault Tolerance

If one machine fails, Spark recreates lost data using lineage (RDD). No need to restart the whole job.

6. High Scalability

Can scale from 1 Machine to 1000+ Machines.

7. Unified Engine

One engine supports Batch Processing, Streaming, SQL, Machine Learning, Graph Processing.

Types of Data Processing

Only two major types.

Batch Processing

Entire data available



Process all together



Generate output

Example: Daily sales report.

Stream Processing

Data keeps coming continuously



Process immediately

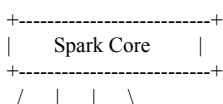


Output continuously

Example: Credit card fraud detection, Stock Market, IoT Sensors.

Spark Ecosystem

Apache Spark



Spark Core

Foundation of Spark. Provides RDD, Scheduling, Memory Management, Fault Tolerance, Task Execution. Everything internally depends on Spark Core.

Spark SQL

Used for SQL Queries, DataFrames, Datasets. Supports SELECT, JOIN, GROUP BY, WINDOW FUNCTIONS.

Structured Streaming

Processes streaming (real-time) data. Examples: Kafka, IoT, Log files.

MLlib

Machine Learning Library. Contains Regression, Classification, Clustering, Recommendation, Feature Engineering.

GraphX

Graph Processing. Example: Social Networks, Road Networks, Recommendation Graphs.

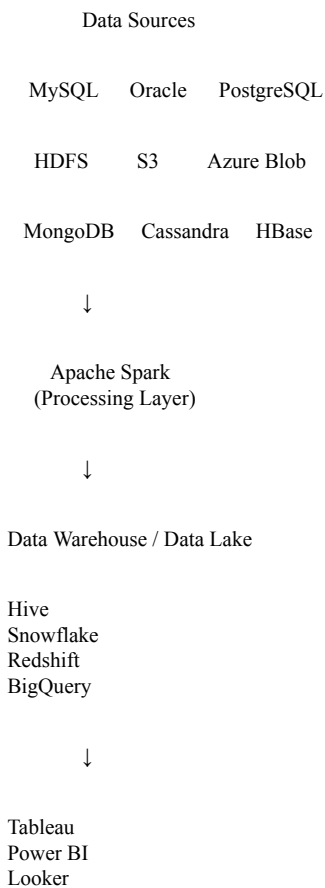
Correction

✗ "Spark only does batch."

Not exactly. Spark Core was originally designed mainly for batch processing. Streaming is handled through Structured Streaming, which is built on top of Spark SQL and Spark Core.

So Spark supports both batch and streaming.

Where Spark Fits in Data Engineering



Spark is not a storage system. Spark is a processing engine.

Spark Replaces What?

Spark mainly replaces Hadoop MapReduce. Not Hadoop itself. HDFS remains. YARN remains. Hive remains. Sqoop remains. Spark simply becomes the processing engine.

Old: Hive → MapReduce

New: Hive → Spark Engine

Similarly, Pig can execute using Spark instead of MapReduce.

Languages Supported

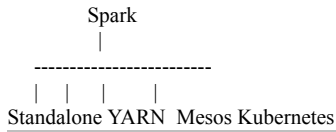
Spark is written in Scala (native language) and Java.

APIs available in Scala, Java, Python (PySpark), R.

Most Data Engineers use PySpark.

Deployment Modes (Very Important)

Spark needs a Cluster Manager.



Standalone

Spark's own cluster manager. Simple. Mostly for learning and small clusters.

YARN

Used in Hadoop clusters. Most common in traditional Big Data companies.

Mesos

General cluster manager. Used less nowadays.

Kubernetes

Modern cloud deployment. Very popular today. AWS, Azure, GCP, Databricks, EMR.

Daemon Processes

Master

Controls the cluster. Allocates resources. Schedules jobs.

Worker

Runs the actual tasks. Stores cache. Executes computations.

Spark Shells

Shell	Purpose
spark-shell	Scala REPL — used by Scala developers
pyspark	Python REPL — used by Data Engineers
spark-submit	Used to execute production applications, e.g. spark-submit app.py

Databricks

Databricks already provides Spark, PySpark, Cluster, Notebook. No installation required.

Installing PySpark Locally (Latest Version)

- 1. Install Java (17 or compatible).
- 2. Install Python (3.10–3.12 is commonly supported by recent PySpark releases).
- 3. Install PySpark:

```
pip install pyspark
```

- 4. Verify installation:

```
pyspark
```

or in Python:

```
import pyspark
print(pyspark.__version__)
```

- 5. Create a Spark session:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
    .appName("Demo") \
    .getOrCreate()
```

For learning, Databricks Community Edition is the easiest option because everything is preconfigured.

One-Line Interview Summary

Apache Spark is an open-source distributed data processing engine that performs parallel computation across a cluster of machines using in-memory processing, making it significantly faster than Hadoop MapReduce while supporting batch processing, SQL, streaming, machine learning, and graph analytics.

B. SPARK ARCHITECTURE — END-TO-END CASE STUDY

Case Study

Suppose Flipkart stores 10 TB of customer transaction data in HDFS. The company wants to calculate: Total sales of every product category.

The Data Engineer writes the following PySpark code:

```
df = spark.read.parquet("/sales")
```

```
result = df.groupBy("category") \
    .sum("amount")
```

```
result.write.parquet("/output")
```

Now let's see what happens internally.

Step 1 — User Submits Spark Job

```
spark-submit \
--master yarn \
--deploy-mode cluster \
--driver-memory 4G \
--executor-memory 8G \
--executor-cores 4 \
--num-executors 20 \
sales.py
```

During submission itself the user specifies: Cluster Manager, Driver Memory, Executor Memory, Number of Executors, Executor Cores, Deployment Mode, Application Name.

The cluster manager reads all these configurations.

Step 2 — Cluster Manager Receives Request

Possible cluster managers: Standalone, YARN, Kubernetes.

The cluster manager's responsibility is only resource management. It does NOT execute Spark code.

It only answers: "Where can I launch Driver?", "Where can I launch Executors?", "Is enough memory available?"

Think of it as a Hotel Receptionist. You ask "I need one conference room and twenty employees." Reception checks availability. Reception doesn't do the work.

Step 3 — Driver Program Starts

The first process created is the Driver Program — this is the brain of Spark.

It contains: SparkSession, SparkContext, DAG Scheduler, Task Scheduler, Catalyst Optimizer (Spark SQL), Memory Manager.

The Driver is NOT processing data. It is coordinating everything.

What Exactly is Driver?

The Driver is responsible for:

- ✓ Reading your code
- ✓ Creating execution plan
- ✓ Asking Cluster Manager for Executors
- ✓ Sending tasks
- ✓ Receiving task results
- ✓ Monitoring Executors
- ✓ Handling failures

Think of Driver as a Project Manager. Executors are Employees.

Step 4 — SparkSession is Created

```
spark = SparkSession.builder \
    .appName("Sales") \
    .getOrCreate()
```

What is SparkSession?

SparkSession is the entry point of Spark. Every Spark application begins from SparkSession.

Using SparkSession we can:

Action	Code
Read Data	spark.read.csv()
Read Hive	spark.sql()
Read Parquet	spark.read.parquet()
Create DataFrames	spark.createDataFrame()
Run SQL	spark.sql()
Access SparkContext	spark.sparkContext

Think of SparkSession as the Main Gate of Spark.

SparkSession vs SparkContext

Before Spark 2.0: SparkContext, SQLContext, HiveContext, StreamingContext were all separate.

From Spark 2.0, everything is merged into SparkSession, which internally creates SparkContext.

SparkSession



SparkContext

What is SparkContext?

SparkContext is the communication bridge between Driver and Cluster Manager. It registers the Spark Application, requests Executors, creates RDDs, maintains connection, schedules jobs.

Without SparkContext, Driver cannot communicate with cluster.

Earlier:

```
sc = SparkContext()
```

Today:

```
spark.sparkContext # returns the existing SparkContext
```

Does SparkContext Connect Driver to Worker Nodes?

Partially yes. More accurately, SparkContext connects the Driver with the Spark Cluster.

- Registers application
- Requests executors
- Communicates with Cluster Manager
- Sends work to executors through Spark's scheduler

It does not directly execute tasks.

Step 5 — Driver Requests Executors

SparkContext tells Cluster Manager:

I need
20 Executors
Each Executor
8 GB Memory
4 CPU Cores

Cluster Manager checks Available Machines, Available Memory, Available CPUs — then launches Executors.

Standalone Example

Suppose we have Master, Worker 1, Worker 2, Worker 3, Worker 4.

Cluster Manager starts:

Executor	Worker
Executor 1	Worker 1
Executor 2	Worker 2
Executor 3	Worker 3
Executor 4	Worker 4

Important: Worker Node $\neq$ Executor. A Worker Node is a machine. One Worker can run multiple Executors if resources allow.

Worker 1

Executor A
Executor B
Executor C

Step 6 — Reading Data

Suppose 10 TB stored in HDFS. HDFS already stores data in blocks.

Block 1 256 MB
Block 2 256 MB
Block 3 256 MB

...

Spark DOES NOT split files randomly. Spark creates partitions based on input blocks (subject to input format and configuration).

Suppose 10 TB $\rightarrow$ 40,000 blocks. Spark creates roughly 40,000 Partitions. Each partition becomes one unit of work.

Block vs Partition

HDFS Block	Spark Partition
Physical storage	Logical processing unit

Usually 1 Block $\rightarrow$ 1 Partition (not always).

Step 7 — Driver Creates DAG

Spark reads groupBy() and sum(). Spark never executes immediately. Instead it creates a DAG (Directed Acyclic Graph).

A DAG is the logical execution plan.

Step 8 — DAG Scheduler Creates Stages

Spark divides Read, Filter, Join, GroupBy, Sort, Write into stages. Stages are separated by shuffle boundaries.

Stage 1
Read
Filter

Stage 2
Shuffle
GroupBy

Stage 3
Write

Step 9 — Task Scheduler Creates Tasks

Suppose 40,000 partitions — then 40,000 Tasks are created.

One partition $\rightarrow$ One Task

Step 10 — Driver Sends Tasks

Driver sends tasks to Executors.

Executor 1: Task 1, Task 2, Task 3
Executor 2: Task 4, Task 5, Task 6

The Driver sends tasks, not raw data. The data is read directly by Executors from HDFS (or S3, etc.) whenever possible.

Does Driver Split Data?

No. This is one of the biggest misconceptions.

Driver NEVER splits data. HDFS already split it into blocks. Spark converts those blocks into partitions. Executors read their assigned partitions. The Driver only assigns which partition each Executor should process.

Data Locality (Very Important)

Suppose Worker 2 already stores Block 15. Spark prefers Executor on Worker 2 to process Block 15 instead of transferring data over the network.

This is called Data Locality. It reduces network traffic and improves performance.

Step 11 — Executors Execute Tasks

Each Executor: Reads Partition, Processes Partition, Stores Intermediate Data (if needed), Returns status/results to Driver.

Executors also cache data when requested:

```
df.cache()
```

Heartbeats

Every Executor periodically sends a heartbeat to the Driver. Heartbeat contains Alive status, Task status, Memory usage, Executor health.

If heartbeats stop, Driver assumes the Executor failed.

Note: The Driver does not send heartbeats to Executors. Executors periodically send heartbeats to the Driver, and the Driver monitors them.

Executor Failure

Suppose Executor 3 crashes — partition 1456 processing stopped. Driver detects Executor Lost. Now what?

Lineage

RDDs remember how they were created.

```
RDD3
↓
map()
↓
RDD2
↓
filter()
↓
RDD1
```

Spark stores this lineage (the transformation history), not copies of every intermediate result. If a partition is lost, Spark recomputes it from its lineage.

Example

Suppose Partition 2500 was lost. Spark recomputes only Partition 2500 — not all 40,000 partitions.

This is why Spark is fault tolerant.

Does Cluster Manager Launch New Executor?

Yes. If resources are available, Driver asks Cluster Manager for another Executor. Cluster Manager launches a new Executor.

The new Executor receives the remaining pending tasks or recomputation work.

Case Study Flow

```
User
↓
spark-submit
↓
Cluster Manager
↓
Launch Driver
↓
SparkSession Created
↓
SparkContext Created
↓
Driver Requests Executors
↓
Cluster Manager Launches Executors
↓
Executors Read Data
↓
Driver Creates DAG
↓
Stages
↓
Tasks
↓
Tasks Sent
↓
Executors Process Partitions
↓
Heartbeats
```

↓
Driver Monitors
↓
Executor Failure
↓
Driver Detects Failure
↓
Lineage Recomputes Lost Partition
↓
Job Completes

Who is the Master?

This depends on context.

1. Standalone Cluster

Master
↓
Cluster Manager

Here the Master is the machine running Spark's standalone cluster manager. It manages workers and resources.

2. YARN

There is no Spark Master. Instead, YARN ResourceManager acts as the cluster manager. Worker machines run NodeManager.

Spark Driver and Executors run inside YARN containers.

3. Kubernetes

There is no Spark Master. Instead, Kubernetes API Server + Control Plane acts as the cluster manager.

The Driver and Executors run as Kubernetes Pods.

How YARN Manages the Cluster

User
|
spark-submit --master yarn
↓
YARN ResourceManager
|
Allocates a container for the Driver
↓
Spark Driver starts
|
SparkContext requests Executors
↓
ResourceManager allocates more containers
↓
NodeManagers launch Executor containers
↓
Executors process HDFS/S3 data

YARN is responsible for: Resource allocation (CPU, memory), Container lifecycle, Scheduling across multiple applications, Monitoring node health.

Spark is responsible for: DAG creation, Task scheduling, Fault recovery using lineage, Query execution.

How Kubernetes Manages the Cluster

User
|
spark-submit --master k8s://...
↓
Kubernetes API Server
|
Creates Driver Pod
↓
Driver Pod starts
|
SparkContext requests Executors
↓

Kubernetes Scheduler creates Executor Pods



Executors read data and execute tasks

Kubernetes handles: Pod creation and placement, Scaling, Restarting failed Pods (depending on policy), Resource isolation.

Spark still manages execution logic, DAGs, tasks, and lineage.

Interview Summary

Q: What is SparkSession?

A: It is the unified entry point to Spark. It provides APIs for DataFrames, Spark SQL, Hive, streaming integration, and internally creates or uses a SparkContext.

Q: What is SparkContext?

A: It is the core execution context that registers the application with the cluster manager, requests executors, creates RDDs, and enables the Driver to communicate with the Spark cluster.

Q: Does the Driver split the data?

A: No. Storage systems such as HDFS already divide data into blocks. Spark creates partitions from the input and schedules tasks against those partitions. Executors read the data directly.

Q: What does the Driver do?

A: The Driver reads the application, creates the DAG, divides it into stages and tasks, requests executors, schedules tasks, monitors execution, and coordinates fault recovery.

Q: What is an Executor?

A: An Executor is a JVM process running on a worker machine. It executes tasks, reads data, caches partitions, stores shuffle data, and sends status updates (heartbeats) to the Driver.

Q: What is lineage?

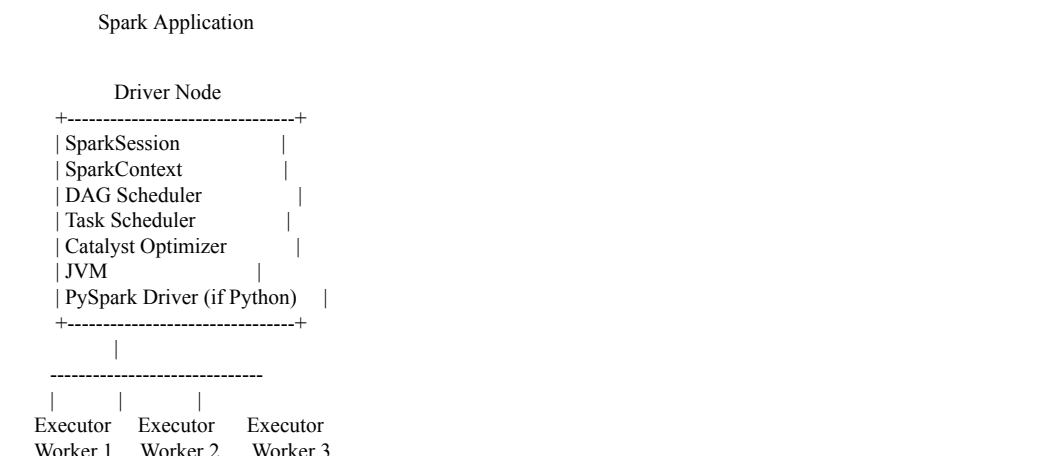
A: Lineage is the record of transformations used to create an RDD or DataFrame. If a partition is lost, Spark uses this lineage to recompute only the missing partition instead of rerunning the entire job.

Q: What is the difference between Standalone, YARN, and Kubernetes?

A: The Spark execution engine remains the same. The difference is only in who manages cluster resources: Standalone uses Spark's built-in Master to manage Workers; YARN uses the Hadoop ResourceManager and NodeManagers to allocate containers for the Driver and Executors; Kubernetes uses its control plane to schedule Driver and Executor Pods. Once running, Spark's internal execution flow (DAG → Stages → Tasks → Executors → Lineage-based recovery) is essentially the same across all three deployment modes.

C. DRIVER NODE VS WORKER NODE — INTERNALS

Architecture Overview



One Driver → Many Executors. Executors run on Worker Nodes.

What Exactly is the Driver Node?

The Driver Node is the brain of a Spark application. It never processes large datasets itself. Instead, it coordinates the entire execution.

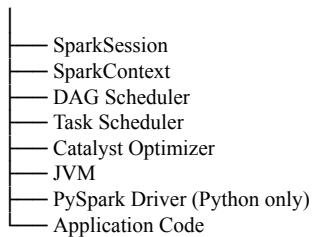
Think of it like a project manager. A project manager doesn't build the house. He assigns work to workers.

Similarly, Driver:

- Reads your Spark code
- Builds execution plan
- Requests resources
- Launches executors
- Sends tasks
- Collects results
- Handles failures
- Monitors execution

Components Inside Driver Node

Driver Node



1. Application Code

This is simply your program. The Driver first executes your application code.

```
df = spark.read.csv("/sales")
```

```
df.groupBy("category") \
  .sum("amount") \
  .show()
```

2. SparkSession

SparkSession is the entry point to Spark. Every DataFrame or Spark SQL operation starts here.

```
spark.read.csv()
spark.sql()
spark.createDataFrame()
```

SparkSession internally creates (or retrieves) SparkContext.

3. SparkContext

SparkContext is the connection between the Driver and the Spark Cluster. Its responsibilities:

- Registering the Spark application
- Talking to the cluster manager (YARN, Kubernetes, Standalone)
- Requesting executors
- Creating RDDs
- Starting Spark jobs

Think of SparkContext as the Driver's communication layer with the cluster.

4. DAG Scheduler

Once an action is called, Spark creates a DAG (Directed Acyclic Graph). The DAG Scheduler analyzes the DAG, splits it into stages, and finds shuffle boundaries.

```
Read
↓
Filter
↓
Map
↓
GroupBy
↓
Write
```

The scheduler determines where stages should begin and end.

5. Task Scheduler

After stages are created, the Task Scheduler creates tasks, assigns tasks to executors, tracks completion, retries failed tasks.

6. Catalyst Optimizer

This exists only for Spark SQL and DataFrames — not for low-level RDD operations.

Catalyst: rewrites SQL, pushes filters down, removes unnecessary columns, reorders joins, optimizes execution.

7. JVM

Spark is written in Scala. Scala runs on the JVM. Therefore, every Driver always has a JVM inside it.

Even if you write Python, Spark internally is executing JVM code.

8. PySpark Driver

This exists only when using Python.

```
df = spark.read.csv(...)
```

You wrote Python. But Spark itself understands Scala/Java. So how does this work? Internally:

```
Python Code
↓
PySpark
↓
Py4J Library
↓
JVM
↓
Spark Engine
```

Py4J is a bridge that lets Python communicate with the JVM. Python is not converted into Java source code. Instead, Python sends commands through Py4J to the JVM, which invokes Spark's Java/Scala APIs.

Is There an Application Master Inside the Driver?

This depends on the cluster manager.

In YARN

There is an ApplicationMaster (AM), but it is not a Spark component. It is a YARN component.

Historically, the Spark Driver often ran inside the ApplicationMaster (especially in cluster mode on older Spark/YARN versions), so people sometimes used the terms together.

Modern Spark separates the concepts: ApplicationMaster belongs to YARN. Driver belongs to Spark.

✗ **So don't say: "Driver contains ApplicationMaster." That's incorrect.**

Worker Node

Worker Node is simply a machine. It provides CPU, Memory, Disk to run Executors.

Worker Node

CPU
RAM
Disk

↓

Executor JVM

A Worker Node can host multiple Executors if it has enough resources.

Executor

Executor is the actual process that performs work.

Executor

```
├── JVM
├── Task Threads
├── Memory
├── Cache
├── Shuffle Manager
└── Block Manager
```

Executors: Read data, Execute tasks, Cache partitions, Store shuffle data, Send heartbeats, Return results.

Why does Executor have JVM?

Spark Engine is written in Scala. Scala runs inside JVM. So every executor is a JVM process.

Does Executor have Python?

Normally, No. Only JVM.

Then How Does PySpark Run?

Suppose your code is:

```
df.withColumn("x", col("salary") * 2)
```

This is a built-in Spark function. Python sends the instruction. The actual execution happens entirely inside the JVM. No Python process is needed on the Executors.

What About Python UDF?

Suppose you write:

```
from pyspark.sql.functions import udf
```

```
@udf
def bonus(x):
    return x * 10
```

Now Spark cannot execute this inside JVM because the logic is written in Python. So Spark starts a separate Python worker process on each Executor.

```
Executor JVM
↓
Python Worker
↓
Execute UDF
↓
Return to JVM
```

This introduces serialization and inter-process communication overhead, which is why Python UDFs are slower.

JVM vs C++

✗ **"JVM is slowly getting converted to C++."**

This is not correct. Spark is still fundamentally a JVM-based engine.

What has changed is that modern Spark uses more native code for performance. Examples:

- Apache Arrow (columnar in-memory format)
- Native Parquet/ORC readers
- Vectorized execution
- Databricks Photon Engine

What is Photon?

Photon is Databricks' high-performance execution engine. Instead of running many SQL operations in JVM bytecode, Photon executes them using highly optimized native C++ code.

Benefits:

- Lower CPU usage
- Faster joins
- Faster aggregations
- Better vectorization
- Lower latency

Important: Photon does not replace all of Spark. Spark still manages DAG, Scheduling, Fault tolerance, Lineage, Resource management. Photon accelerates the execution of supported SQL/DataFrame operators.

Databricks

Databricks is a managed Spark platform. Instead of manually installing Java, Hadoop, Spark, Cluster Manager, Databricks manages everything.

Think of it as a Managed Spark Platform.

Without Databricks

Linux
↓
Java
↓
Spark
↓
Hadoop
↓
Configure Cluster
↓
Start Driver
↓
Start Workers
↓
Monitor Logs

A lot of manual work.

With Databricks

You simply create a cluster. Databricks automatically:

- Creates VMs
- Installs Spark
- Configures networking
- Starts Driver
- Starts Executors
- Configures storage access
- Enables monitoring
- Handles auto-scaling (if enabled)

You focus only on writing Spark code.

Why Don't We Create SparkSession in Databricks?

Normally:

```
spark = SparkSession.builder \  
  .appName("Sales") \  
  .getOrCreate()
```

In Databricks, when you start a cluster, the platform automatically creates a global SparkSession named spark.

So inside a notebook you can immediately write:

```
df = spark.read.parquet("/mnt/data")
```

without creating it yourself. Internally, Databricks has already:

- 1. Started the Driver JVM.
- 2. Created a SparkContext.
- 3. Created a SparkSession.
- 4. Exposed it as the notebook variable spark.

So you're using the same Spark APIs, just without the setup code.

Lazy Evaluation

One of Spark's biggest optimizations.

```
df = spark.read.csv("/sales")
```

```
df = df.filter(col("amount") > 100)
```

```
df = df.select("customer", "amount")
```

```
df = df.groupBy("customer").sum()
```

```
df.show()
```

Notice: four transformations, one action. Spark does not execute the transformations immediately. Instead,

it records them.

Think of it like making a to-do list.

Read
↓
Filter
↓
Select
↓
GroupBy

Nothing has run yet.

Why?

Because Spark wants the entire picture before executing. If it executed line by line, it would miss optimization opportunities.

Action

An Action tells Spark: "Now execute the plan."

show()
collect()
count()
take()
write()
save()

The moment Spark sees an Action, execution begins.

Internal Flow

Transformation
↓
Transformation
↓
Transformation
↓
Transformation
↓
Action
↓
DAG Created
↓
Optimization
↓
Stages
↓
Tasks
↓
Execution

Does Spark Execute Transformations One by One Exactly as Written?

Not necessarily. Spark first builds a logical plan and then optimizes it.

However, it's important to be precise:

- Spark preserves the logical meaning of your program.
- It does not arbitrarily move transformations around if that would change the result.
- It rewrites the plan only when it can prove the result stays the same.

For example, Spark may push a filter closer to the data source. Instead of:

Read
↓
Select
↓
Filter

it may optimize to:

Read
↓
Filter
↓
Select

This is called predicate pushdown, and it reduces the amount of data read.

Spark may also:

- Remove unused columns (column pruning)
- Combine adjacent projections
- Reorder joins (when safe and beneficial)
- Eliminate redundant operations

What Happens After Action?

Logical Plan
↓
Optimized Logical Plan
↓
Physical Plan
↓
DAG
↓
Stages
↓
Tasks
↓
Execution

Explain Command

You can inspect Spark's plans using:

```
df.explain()  
or  
df.explain(True)
```

The extended version shows multiple stages of planning. Typical output contains:

Parsed Logical Plan — your query exactly as Spark parsed it.

↓ Analyzed Logical Plan — Spark resolves tables, columns, and data types.

↓ Optimized Logical Plan — Catalyst applies optimizations such as predicate pushdown and column pruning.

↓ **Physical Plan — the actual execution strategy Spark will use (for example, broadcast hash join or sort-merge join).**

Should We Read explain() From Bottom to Top?

For interviews, the best approach is:

Read it top to bottom to understand how Spark transforms your query:

- 1. Parsed Logical Plan
- 2. Analyzed Logical Plan
- 3. Optimized Logical Plan
- 4. Physical Plan

Within the Physical Plan tree, execution starts from the leaf nodes (data sources) and flows upward to the root. That's why people often say "read the physical tree from bottom to top."

Both viewpoints are useful, but they describe different things.

Interview Summary

Q: What is the Driver Node?

A: The Driver Node is the coordinator of a Spark application. It runs the application code, creates the execution plan, requests executors, schedules tasks, monitors execution, and handles failures.

Q: What is the Worker Node?

A: A Worker Node is a machine that provides CPU and memory resources. It hosts one or more Executor JVM processes.

Q: What is an Executor?

A: An Executor is a JVM process that executes tasks, reads partitions, caches data, manages shuffle files, and reports status to the Driver.

Q: Why are Python UDFs slower?

A: Because each Executor must invoke a separate Python worker process and exchange data between the JVM and Python, adding serialization and communication overhead.

Q: What is Lazy Evaluation?

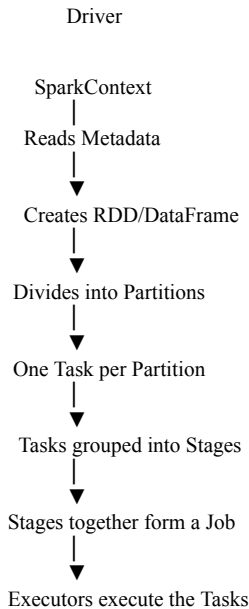
A: Spark records transformations without executing them immediately. When an action is encountered, Spark optimizes the entire plan, creates stages and tasks, and then executes them.

Q: What does explain() show?

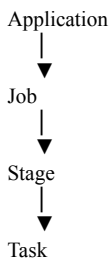
A: It shows how Spark interprets and optimizes your query: Parsed Logical Plan → Analyzed Logical Plan → Optimized Logical Plan → Physical Plan. This is one of the best tools for understanding Spark's execution strategy.

D. RDD, LINEAGE, PARTITIONS & SHUFFLE INTERNALS

Spark Execution Flow (Big Picture)



Remember this hierarchy forever:



What is RDD?

RDD stands for Resilient Distributed Dataset.

Term	Meaning
Resilient	Can recover lost data
Distributed	Stored across multiple executors
Dataset	Collection of records

Interview Definition: RDD is Spark's immutable, fault-tolerant distributed collection of data divided into logical partitions and processed in parallel.

Why is RDD Immutable?

```
df = spark.read.csv("employees.csv")
```

Internally: RDD-1 (1000 Records)

Now filter:

```
df2 = df.filter("salary > 50000")
```

Does Spark modify df? No.

RDD-1 (Original)

↓ Filter
RDD-2 (Filtered)

Again:

```
df3 = df2.select("name", "salary")
```

RDD-1 → RDD-2 → RDD-3

Every transformation creates a brand-new RDD (or DataFrame backed by a new logical plan). The original remains unchanged. This chain is called Lineage.

What is Lineage?

Lineage is simply the history of how an RDD (or DataFrame) was created.

```
df = spark.read.csv("sales.csv")
```

```
df2 = df.filter("country='India'")
```

```
df3 = df2.select("product", "sales")
```

```
df4 = df3.groupBy("product").sum("sales")
```

Spark remembers:

```
Read CSV
  ↓
Filter India
  ↓
Select Columns
  ↓
Group By Product
  ↓
RDD4
```

Spark does not execute immediately because transformations are lazy. It builds this execution plan (DAG) first.

Why is RDD Called Resilient?

RDD
P1
P2
P3
P4

Executor-2 crashes — Lost: P3.

Spark does NOT restore from a backup copy. Instead, it asks: "How was P3 created?" It checks the lineage.

```
Read CSV
  ↓
Filter
  ↓
Map
  ↓
Partition 3
```

It reruns only those transformations for Partition 3. This is why RDD is called Resilient.

Important Interview Point: Spark does not replicate RDDs for fault tolerance. HDFS replicates storage blocks. Spark recomputes lost partitions using lineage.

Logical Partitions

Suppose Employee.csv has 1000 rows. Spark creates:

RDD
P1 → 250 rows
P2 → 250 rows
P3 → 250 rows
P4 → 250 rows

Each partition becomes one task:

Task 1 → P1

Task 2 → P2
Task 3 → P3
Task 4 → P4

Partition Creation

HDFS

10GB File
↓
80 HDFS Blocks
↓
80 Spark Partitions

Generally: 1 Block $\approx$ 1 Partition

S3

No HDFS blocks. Spark creates partitions using byte ranges:

0-128MB
128-256MB
256-384MB

Each becomes a partition.

JDBC

No automatic partitioning. Without partitioning:

Database
↓
One Connection
↓
One Partition

With:

```
spark.read.jdbc(  
  url=url,  
  table="employees",  
  column="id",  
  lowerBound=1,  
  upperBound=1000000,  
  numPartitions=10  
)
```

Spark creates id 1–100000, id 100001–200000, ... 10 partitions.

How Partitions are Distributed

RDD: P1 P2 P3 P4 P5 P6

Spark distributes:

Worker	Partitions
Worker-1 (Executor)	P1, P2
Worker-2 (Executor)	P3, P4
Worker-3 (Executor)	P5, P6

RDD remains one logical dataset. Only partitions are distributed.

Number of Partitions

```
df = spark.read.csv("employees.csv")
```

```
print(df.rdd.getNumPartitions())
```

Output: 4

Repartition()

Used to increase or redistribute partitions.

```
df2 = df.repartition(10)
```

Before: P1 P2 P3 P4 → After: P1...P10

How?

Spark shuffles data.

Before: After repartition(4):
P1: A B C P1: A D
P2: D E F P2: B E
 P3: C
 P4: F

Records move between partitions. This movement is called Shuffle.

Shuffle involves: Reading data from executors, Sending data over the network, Writing temporary files, Reading them again. Hence it is expensive.

Read
↓
Filter
↓
Repartition
=====

Shuffle
=====

↓
Next Stage

Every shuffle creates a stage boundary.

Coalesce()

Used mainly to reduce partitions efficiently.

Current: 10 partitions. Need: 3 partitions.

`df2 = df.coalesce(3)`

Spark simply merges nearby partitions:

P1 + P2 → P1
P3 + P4 → P2
P5 + P6 → P3

Usually no shuffle occurs. Therefore much faster.

✗ coalesce() avoids shuffle by default

However, if you explicitly use `coalesce(3, shuffle=True)` (or the Scala equivalent), then shuffle can happen. In PySpark, the commonly used `coalesce(numPartitions)` performs a narrow merge without shuffle.

Repartition vs Coalesce

Repartition	Coalesce
Increase or decrease partitions	Mainly decrease partitions
Full shuffle	No shuffle by default
Expensive	Cheap
Evenly redistributes data	Merges existing partitions, may create uneven sizes
Wide transformation	Narrow transformation (default behavior)

Transformations

Transformations are lazy. Nothing executes until an action is called. Two types.

Narrow Transformation

Each output partition depends on only one input partition.

`df.filter()`
`df.select()`
`df.withColumn()`
`df.map()`

P1 → Filter → P1
P2 → Filter → P2

No communication between partitions. No shuffle. Fast.

Wide Transformation

Output partition depends on multiple input partitions.

`groupBy()`
`join()`
`distinct()`
`orderBy()`
`dropDuplicates()`
`repartition()`
`reduceByKey()`

Suppose P1 has India, USA, India and P2 has USA, India, USA. Now `groupBy("country")` — Spark cannot aggregate without bringing all "India" rows together and all "USA" rows together. It performs shuffle.

After shuffle: P1 = India, India, India; P2 = USA, USA, USA. Now aggregation starts.

This is why `groupBy()` is a wide transformation.

Shuffle

Shuffle means moving records from one partition to another across the cluster.

Before: After shuffle:
P1: India, USA P1: India, India
P2: USA, India P2: USA, USA

Rows moved between executors. This involves network I/O and disk I/O, making it expensive.

What is Exchange in Spark?

When you view a physical execution plan (`df.explain()`), you'll often see:

Exchange hashpartitioning(...)

Exchange is Spark's physical operator for shuffle.

Exchange = Shuffle

Default Shuffle Partitions

For many wide operations (such as `groupBy`, `join`, `distinct`), Spark SQL uses:

`spark.sql.shuffle.partitions`

Default value: 200

So after a shuffle, Spark typically creates 200 partitions unless you change this setting.

Jobs, Stages and Tasks

This is one of the most important interview topics.

Job

A Job is created whenever an Action is called.

```
df.filter("salary>50000") \
  .groupBy("dept") \
  .count() \
  .show()
```

Action `show()` creates one Job.

Stage

A Job is divided into stages. Stages are separated by shuffle boundaries, not by "all narrow transformations" or "all actions."

```
Job
↓
Stage 1
Read
↓
Filter
↓
Select
↓
Shuffle
```

Then:

```
Stage 2
GroupBy Aggregation
↓
Count
↓
Show
```

Only one job, but two stages because `groupBy()` introduces a shuffle.

Task

Each partition becomes one task within a stage.

```
Stage: 4 partitions
↓
Task 1, Task 2, Task 3, Task 4
```

Each task processes exactly one partition.

Complete Example

```
df = spark.read.csv("sales.csv")
```

```
df2 = df.filter("sales>100")
```

```
df3 = df2.select("country","sales")
```

```
df4 = df3.groupBy("country").sum("sales")
```

```
df4.show()
```

Action: show()

↓

One Job

↓

Stage 1

Read

↓

Filter

↓

Select

↓

Shuffle

↓

Stage 2

GroupBy

↓

Aggregation

↓

Result

↓

Driver

If there are 4 partitions before the shuffle:

- Stage 1 → 4 tasks (one per partition).
- After the shuffle, suppose Spark creates the default 200 shuffle partitions.
- Stage 2 → 200 tasks (unless `spark.sql.shuffle.partitions` is changed).

Interview Summary

Concept	Summary
RDD	Immutable distributed collection of data, divided into logical partitions, fault tolerant through lineage
Lineage	Stores the sequence of transformations used to build an RDD/DataFrame; used to recompute only lost partitions
Partition	Logical chunk of an RDD/DataFrame; one partition corresponds to one task within a stage
Repartition	Wide transformation; performs a full shuffle; used to increase or evenly redistribute partitions
Coalesce	Narrow transformation by default; mainly used to reduce partitions; avoids shuffle unless explicitly requested
Narrow Transformation	No shuffle; each output partition depends on a single input partition
Wide Transformation	Requires shuffle; output partitions depend on multiple input partitions
Shuffle	Movement of data across partitions/executors; expensive; appears as Exchange in the physical plan
Job	Created by an action (show, count, collect, write, etc.)
Stage	A group of tasks that can execute without a shuffle; new stages begin at shuffle boundaries
Task	The smallest unit of execution; exactly one task processes one partition in a given stage

This model — Application → Job → Stage → Task → Partition — is the mental framework interviewers expect when discussing Spark execution.

E. SPARK JOINS

What is a Wide Transformation?

A wide transformation is an operation where Spark needs data from multiple partitions to produce the result.

Examples: `join()`, `groupBy()`, `reduceByKey()`, `distinct()`, `orderBy()`.

Unlike narrow transformations, data must move across the network. This movement is called Shuffle.

Example

Executor 1	Executor 2
Partition 0:	Partition 0:
101 Alice	101 Sales
102 Bob	104 HR

Partition 1:	Partition 1:
103 John	102 IT
104 David	103 Finance

```
df1.join(df2,"id")
```

Can Spark directly join? No. Because 101 is in Executor1-Partition0, while its match is in Executor2-Partition0 — different executors. Spark cannot know in general where matching keys live.

Therefore Spark first rearranges data. This rearrangement is called Shuffle.

Shuffle

Shuffle means moving records across executors so that identical keys reach the same partition.

Spark uses `Hash(key)` to decide which partition each key belongs to.

Example — suppose `spark.sql.shuffle.partitions = 4`. Spark computes `Partition = hash(key) % 4`:

```
hash(101)%4 = 1
hash(102)%4 = 3
hash(103)%4 = 0
hash(104)%4 = 2
```

Now every dataframe performs the same calculation. Both dataframes move rows.

```
Partition0: 103 John, 103 Finance
Partition1: 101 Alice, 101 Sales
Partition2: 104 David, 104 HR
Partition3: 102 Bob, 102 IT
```

Both rows having same key reached same partition. Now join becomes easy.

Why Shuffle?

Spark can only process one partition independently. If the same key is in different executors, they cannot join.

```
Same key
↓
Same partition
↓
Same executor
↓
Join possible
```

`spark.sql.shuffle.partitions = 200`

Many notes say it "creates 200 partitions." What actually happens?

Suppose your dataframe initially has 8 partitions. After join, Spark performs shuffle. Instead of keeping 8, it creates 200 shuffle partitions (default).

Before: P0, P1, P2, ..., P7

After shuffle: P0, P1, P2, ..., P199

Why? To increase parallelism. Each partition becomes one task → 200 partitions → 200 tasks → can run on many cores.

Are 200 Partitions Always Created?

No. Only during shuffle operations.

Triggers shuffle (~200 partitions)	Does NOT trigger shuffle
<code>join()</code> , <code>groupBy()</code> , <code>distinct()</code> , <code>sort()</code> , <code>reduceByKey()</code>	<code>select()</code> , <code>filter()</code> , <code>map()</code> , <code>flatMap()</code>

Spark Join Types

Spark mainly performs: 1. Sort Merge Join, 2. Shuffle Hash Join, 3. Broadcast Join.

1. Sort Merge Join (Default)

This is Spark's default join. Used for Large table + Large table.

Customer: 101 Alice, 102 Bob, 103 John
Orders: 102 Laptop, 101 Mobile, 103 TV

Step 1 — Shuffle both tables:

Partition1: 101 Alice, 101 Mobile
Partition2: 102 Bob, 102 Laptop
Partition3: 103 John, 103 TV

Step 2 — Sort inside every partition (before: 103,101,102 → after: 101,102,103). Both tables are sorted.

Step 3 — Merge, exactly like merging two sorted arrays. Pointer1 → 101, Pointer2 → 101 → Match → Join.
Move both pointers, repeat.

This merge step is very efficient because both inputs are already sorted.

Why sort? Without sorting, Spark would have to search repeatedly for matching keys. Sorting enables a linear merge.

Characteristics: ✓ Shuffle both tables, ✓ Sort both tables, ✓ Merge. Good for Large tables.

Interview Answer: Sort Merge Join is Spark's default join strategy for joining two large datasets. Spark first shuffles both datasets so that identical keys are placed in the same partition. Then it sorts each partition by the join key and finally performs a merge operation similar to merging two sorted arrays. Although sorting adds overhead, it scales well for very large datasets.

2. Shuffle Hash Join

Now imagine one table is 5 GB, the other is 200 MB. Sorting is expensive. Instead, Spark builds a Hash Table.

Think of it like a dictionary:

student = {101:"Alice", 102:"Bob", 103:"John"}

student[102] # ~O(1) lookup

Spark uses the same idea.

Small table:
101 Sales, 102 IT, 103 HR

→ Hash Table:
101→Sales, 102→IT, 103→HR

Now read big table row by row: 101 → Hash lookup → Sales. Join complete. No sorting needed.

Does Spark Create One Hash Table?

No. This is where many people get confused. Spark creates one hash table per partition.

Partition0: 101 Sales, 102 IT → Hash Table: 101→Sales, 102→IT
Partition1: 103 HR, 104 Finance → Hash Table: 103→HR, 104→Finance

Each executor builds hash tables only for the partitions it owns. Because partitions are processed independently — Spark never creates one huge hash table for the entire cluster.

Shuffle
↓
Smaller side
↓
Build hash table
↓
Read larger table
↓
Hash lookup
↓
Join

Characteristics: Shuffle both tables, No sorting, Uses hash lookup. Fast when one shuffled side is significantly smaller.

Interview Answer: In Shuffle Hash Join, Spark first shuffles both datasets so matching keys end up in the same partition. Instead of sorting, Spark builds a hash table for the smaller partition and probes it with rows from the larger partition. Since hash lookups are very fast, this strategy avoids sorting overhead, but it requires enough memory to hold the smaller partition's hash table.

3. Broadcast Join

This is the fastest join. Used when one table is very small.

Example: Employees = 100 Million rows, Departments = 20 rows. Should Spark shuffle 100 million rows? No.

Instead, Spark sends 20 rows to every executor.
Driver broadcasts Departments (20 rows) to Executor1, Executor2, Executor3

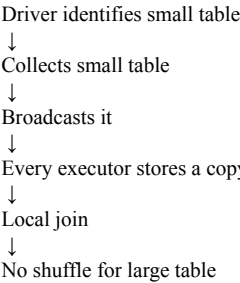
Now each executor already has the entire small table in memory. Each executor joins locally. No shuffle of the large table is required.

Why is it Called Broadcast?

Because Driver sends one copy → Every executor receives it. Like broadcasting a TV signal to all receivers.

Where is it Stored?

The small broadcasted table is kept in the executors' execution memory for the duration of the join. The driver coordinates creating the broadcast variable, but executors use their local in-memory copy during the join.



Characteristics: No shuffle of the large table, No sorting, Fastest join. Requires the small table to fit comfortably in memory.

Interview Answer: Broadcast Join is used when one dataset is small enough to fit in memory. Spark sends a copy of the small dataset from the driver to every executor. Each executor then joins its local partitions of the large dataset with the broadcasted copy, eliminating the expensive shuffle of the large dataset. This is usually the fastest join strategy.

Comparison

Feature	Sort Merge Join	Shuffle Hash Join	Broadcast Join
Shuffle	Yes (both tables)	Yes (both tables)	No shuffle for the large table
Sorting	Yes	No	No
Hash Table	No	Yes (per partition of smaller side)	Yes (broadcasted table used for lookups)
Best For	Large vs Large	Large vs Medium/Smaller	Large vs Very Small
Speed	Moderate	Faster than sort-merge when applicable	Fastest

Interview Summary

- Wide transformation: Operations like join, groupBy, and distinct require data movement across the cluster.
- Shuffle: Redistributes records using a hash of the key so that identical keys land in the same partition.
- spark.sql.shuffle.partitions: Controls the number of partitions created after a shuffle (default is 200), which determines the number of parallel tasks.
- Sort Merge Join: Shuffles both datasets, sorts each partition by the join key, and then merges the sorted data. Default strategy for large datasets.
- Shuffle Hash Join: Shuffles both datasets, builds a hash table for the smaller partition on each executor, and performs fast hash lookups instead of sorting.
- Broadcast Join: Broadcasts a small table to every executor, allowing local joins without shuffling the large table. This is the fastest strategy when one table is small enough to fit in memory.

F. BROADCAST VARIABLES — DEEP DIVE

Imagine You Own a Company

You have 3 offices (executors).
Driver (Manager)
|

| | |
Executor1 Executor2 Executor3

Suppose each office has employee data:

Executor 1: 101 Alice, 102 Bob
Executor 2: 103 John, 104 David
Executor 3: 105 Sam, 106 Mike

Now you also have a tiny table — Department Table, only 6 rows:

101 Sales, 102 IT, 103 HR, 104 Finance, 105 Marketing, 106 Admin

Option 1: Shuffle Join

Without broadcast, Spark says: "Move employees around until matching department records are together." This means lots of data travels over the network. Large amount of movement. Slow.

Option 2: Broadcast Join

Spark thinks: "Wait... the Department table is only 6 rows. Instead of moving millions of employee records, why not copy these 6 rows everywhere?"

Driver: Department Table

↓ Copies it

Executor1: Employee Table + Department Table (copy)

Executor2: Employee Table + Department Table (copy)

Executor3: Employee Table + Department Table (copy)

Every executor now has its own employee partition and one complete copy of Department table. No employee movement. No shuffle.

Is Every Executor Having One Copy of the Small Table?

YES.

If there are 10 executors, Spark creates 10 copies. One inside every executor. Not shared. Every executor owns its own copy.

Does Driver Release the Table After Execution?

Yes. Think of broadcast as temporary.

Driver

|
Broadcasts table

|
Executors use it

|
Join completes

|
Job finishes

Spark automatically frees that memory when it is no longer needed, unless another stage is still using it. So it is not permanently stored.

What is a Broadcast Variable?

This is the most important concept.

Normally, suppose your code is:

```
countries = {  
  1: "India",  
  2: "USA",  
  3: "UK"  
}
```

Imagine 100 tasks. Without broadcast, every task sends countries again... and again... 100 times. Huge waste.

Instead Spark says: send countries once.

Driver: countries dictionary

↓ Send once

Executor 1: Stores countries

Executor 2: Stores countries

Executor 3: Stores countries

Now every task running inside Executor 1 simply reads Executor1 Memory → countries. No network. No repeated sending.

That stored object is called a Broadcast Variable.

Without broadcast: Task1 gets table, Task2 gets table, Task3 gets table, Task4 gets table — same table copied

many times.

With broadcast: one copy in Executor Memory (Department Table); Task1, Task2, Task3, Task4 all use the same copy.

Where is Broadcast Variable Stored?

Inside the executor's memory.

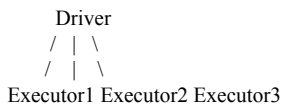
Executor

Task
Task
Task
Broadcast Variable
Execution Memory

Every task running on that executor reads the same broadcast variable.

Then Why Does the Driver Broadcast?

Because only the Driver knows the execution plan. Executors don't communicate directly with each other.



Driver is the coordinator. So Driver says: "Everyone, take this small table."

Does the Driver Keep Another Copy?

Yes. Initially, the Driver has the original object because it created or received it. Then it serializes (converts into bytes) and sends copies to every executor.

After the job, Spark cleans up both the driver-side broadcast metadata and the executor-side copies when they are no longer needed.

Complete Broadcast Join Flow

Employee Table (100 GB)
Executor1, Executor2, Executor3

Small Department Table (10 MB) — Driver

↓ Broadcast

Executor1: Employee Partition + Department Table

Executor2: Employee Partition + Department Table

Executor3: Employee Partition + Department Table

↓

Each executor performs the join locally.

↓

Result returned.

↓

Broadcast variable is removed from memory when the job completes or when Spark determines it is no longer needed.

Interview Answer

A broadcast variable is a read-only copy of a small dataset that Spark sends once from the Driver to every Executor. Each Executor stores one copy in memory, and all tasks running on that Executor reuse that same copy. During a Broadcast Join, Spark broadcasts the small table to every Executor, allowing each Executor to join its local partition of the large table without shuffling the large dataset. Once the job is finished and the broadcast variable is no longer needed, Spark releases the memory automatically.

G. CATALYST OPTIMIZER — END-TO-END

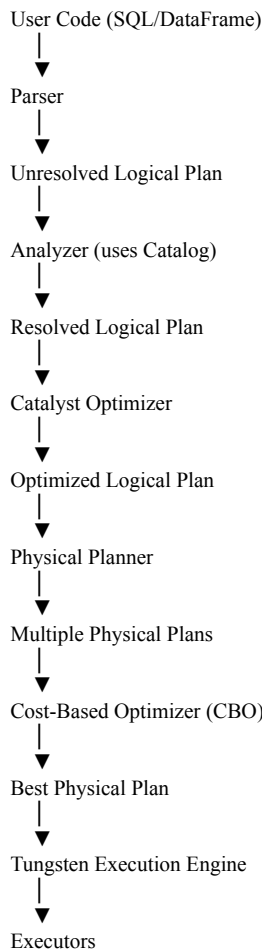
Big Picture

When you write Spark SQL or DataFrame code, Spark does not execute it immediately. Instead, Spark first tries to understand your code, optimize it, and then generate the fastest execution plan.

The engine responsible for this is called the Catalyst Optimizer. Finally, another component called Tungsten executes the optimized plan efficiently.

Developer





Step 1 — User Writes Code

```
df = spark.read.csv("employees.csv", header=True)
```

```
result = (  
    df.filter(df.salary > 50000)  
    .groupBy("department")  
    .avg("salary")  
)
```

Nothing is executed yet. Spark only records: "The user wants to read data, filter rows, group data, and calculate average." No computation happens. This is because Spark uses Lazy Evaluation.

Step 2 — Parser

The parser checks whether your SQL/DataFrame syntax is valid.

```
SELECT name  
FROM employee  
WHERE salary > 50000
```

Parser converts this into Spark's internal tree representation. It understands SELECT, FROM, WHERE, GROUP BY, JOIN.

At this stage Spark only understands the structure. It does not know whether table exists, columns exist, or datatype is correct.

Output: Unresolved Logical Plan

What is an Unresolved Logical Plan?

Spark knows "I need these operations." But it doesn't know where tables exist, where columns exist, datatype, or relation between tables.

```
SELECT salary  
FROM employees
```

Spark only knows:

```
Project(salary)  
|  
employees
```

It doesn't know whether employees exists.

Step 3 — Analyzer

Now Spark starts validating everything. This stage uses the Catalog.

What is Catalog?

Think of Catalog as Spark's dictionary or phone book. It stores metadata.

Catalog does NOT store actual data. It stores information ABOUT data.

Catalog Contains

#	Item	Example
1	Database Names	sales_db, employee_db, finance_db
2	Table Names	employees, customers, orders
3	Column Names	id, name, salary, department
4	Column Datatypes	salary→Integer, name→String, dob→Date
5	Partition Information	Partitioned by country, year, month
6	File Location	s3://company-data/employees/, HDFS path, or Delta table location
7	Statistics	Number of rows, file size, average row size, distinct values, null count
8	Table Format	Parquet, ORC, CSV, Delta, Iceberg
9	Temporary Views	df.createOrReplaceTempView("emp")
10	Functions	count(), avg(), sum() — plus user-defined functions like calculate_bonus()

Statistics are later used by the Cost-Based Optimizer.

Simple Analogy

Imagine a library. Actual books = Data. Library catalog = Metadata. Catalog says Book Name, Shelf Number, Author, Pages, Genre — but the catalog doesn't contain the book itself. Spark Catalog is exactly like that.

Analyzer Uses Catalog

```
SELECT salary
FROM employees
```

Analyzer: Does employees table exist? → Catalog: Yes

Analyzer: Does salary column exist? → Catalog: Yes

Analyzer: Datatype? → Catalog: Integer

Everything verified. Now Spark creates a Resolved Logical Plan.

What is Resolved Logical Plan?

Everything is now validated. Spark knows: table exists ✓, columns exist ✓, datatypes ✓, relations ✓. Still, no optimization yet.

Step 4 — Catalyst Optimizer

Now Spark asks: "Can I make this query faster?" This is where optimization happens. Output: Optimized Logical Plan.

Optimizations Performed

1. Predicate Pushdown

Instead of: Read all rows → Filter rows
Spark changes to: Filter while reading

Much less data.

2. Column Pruning

Suppose table has 100 columns. User only needs name, salary. Spark reads only name, salary.

3. Constant Folding

WHERE age > 10+20
Spark changes: 10+20 → 30

4. Remove Unnecessary Operations

Example: Filter(True). Spark removes it.

5. Combine Filters

Instead of: Filter → Filter → Filter
Spark creates: One Filter

6. Join Reordering

Suppose A, B, C — Spark changes join order if another order is faster.

7. Null Propagation

Spark simplifies expressions involving NULL.

Step 5 — Physical Planner

Now Spark knows "What." Now it decides "How." This is Physical Planning.

Instead of saying "Join Employees and Departments", Spark decides Broadcast Join, or Sort Merge Join, or Shuffle Hash Join.

Multiple Physical Plans

Spark does not generate only one plan.

Plan 1: Broadcast Join

Plan 2: Sort Merge Join

Plan 3: Shuffle Hash Join

Plan 4: Nested Loop Join

Step 6 — Cost-Based Optimizer (CBO)

Now Spark compares all physical plans. Question: which one is cheapest?

Cost means: CPU usage, Memory usage, Disk I/O, Shuffle amount, Network transfer, Estimated execution time.

Example — Employee: 100 Million rows, Department: 10 rows.

Join Option	Estimated Cost
Broadcast Join	20
Sort Merge Join	80
Shuffle Hash Join	65

Spark chooses Broadcast Join — because lowest cost.

Where Does CBO Get Information?

From Catalog Statistics — Row count, Table size, Distinct values, Null count, Average row size.

These help estimate cost. Without statistics, Spark falls back to rule-based decisions.

Step 7 — Tungsten Execution Engine

Now Spark has selected the best physical plan. Tungsten executes it.

Responsibilities include:

- Generating optimized JVM bytecode (Whole-Stage Code Generation)
- Efficient memory management (off-heap memory where possible)
- CPU-efficient processing
- Fast binary data format (UnsafeRow)
- Reduced object creation and garbage collection

The goal is to execute the chosen physical plan as efficiently as possible.

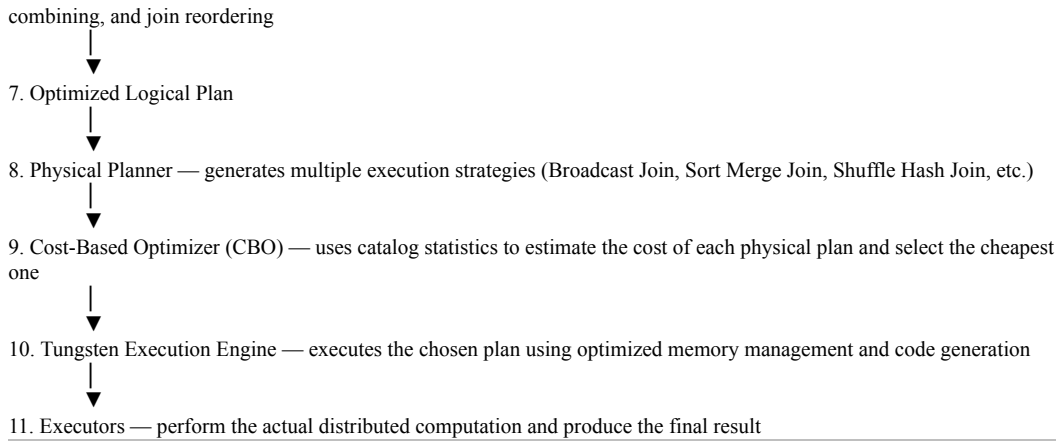
Step 8 — Executors

Finally, the Driver sends tasks to Executors. Executors: Read data, Perform filter, Perform joins, Perform aggregations, Perform shuffle if needed, Return results or write output.

This is where the actual computation happens.

Complete Flow (Interview Summary)

1. Developer writes SQL/DataFrame code
2. Parser — checks syntax
3. Unresolved Logical Plan — knows operations, but not whether tables/columns exist
4. Analyzer — uses Catalog metadata to validate tables, columns, datatypes, functions, and relationships
5. Resolved Logical Plan — fully validated query
6. Catalyst Optimizer — applies logical optimizations such as predicate pushdown, column pruning, constant folding, filter



Interview Answer (2-Minute Version)

"When we write Spark SQL or DataFrame code, Spark doesn't execute it immediately because of lazy evaluation. The parser first checks the syntax and creates an unresolved logical plan. The Analyzer then consults the Catalog, which stores metadata such as database names, table names, column schemas, datatypes, file locations, partitions, functions, and table statistics, to validate the query and produce a resolved logical plan. The Catalyst Optimizer applies rule-based optimizations like predicate pushdown, column pruning, constant folding, filter merging, and join reordering to generate an optimized logical plan. The Physical Planner then creates multiple physical execution plans using different algorithms such as Broadcast Join or Sort-Merge Join. The Cost-Based Optimizer compares these plans using catalog statistics and chooses the one with the lowest estimated cost. Finally, the Tungsten execution engine executes the selected plan efficiently, and the executors perform the actual distributed processing across the cluster."

H. CATALYST OPTIMIZER — SQL vs DATAFRAME vs RDD

✗ Catalyst Optimizer is NOT only for SQL

It is used for both Spark SQL and PySpark DataFrame APIs. The only thing it doesn't optimize is the low-level RDD API.

Which APIs Use Catalyst?

API	Uses Catalyst?	Uses Catalog?
SQL	✓ Yes	✓ Yes
PySpark DataFrame	✓ Yes	✓ Yes
Dataset (Scala/Java)	✓ Yes	✓ Yes
RDD	✗ No	✗ No

These two pieces of code are almost identical from Spark's point of view:

SQL

```
SELECT department,
AVG(salary)
FROM employees
WHERE salary > 50000
GROUP BY department;
```

PySpark

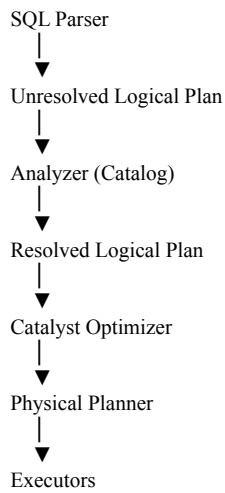
```
df.filter(col("salary") > 50000) \
.groupBy("department") \
.avg("salary")
```

Although the syntax is different, Spark converts both into the same internal logical plan.

How SQL Works

```
SELECT *
FROM employees
WHERE salary > 50000;
```

Flow:
SQL Query
↓

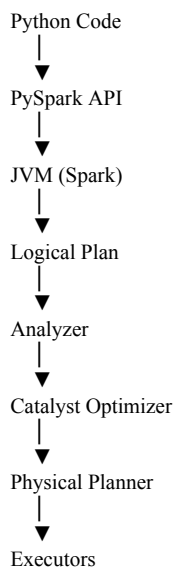


How PySpark DataFrame Works

```
df = spark.read.parquet("/data/employees")
```

```
df.filter(col("salary") > 50000)
```

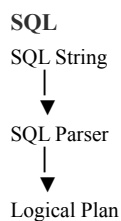
Many people think Catalyst is skipped here. It is not.



The only extra step is that PySpark code is sent from the Python process to the JVM, because Spark itself is written in Scala/Java.

Then What is the Difference?

The difference is only in how the logical plan is created.



PySpark

```
df.select("name")
```

↓

Logical Plan Node

↓

Logical Plan

No SQL grammar needs to be parsed for the DataFrame API — every DataFrame operation builds the logical plan directly.

Example

```
df = spark.read.csv("employee.csv")
```

```
df = df.filter(col("salary") > 50000)
```

```
df = df.select("name", "salary")
```

Spark internally builds something like:

```
Project(name,salary)
  |
Filter(salary > 50000)
  |
CSV(employee.csv)
```

That is the logical plan. Catalyst then optimizes it exactly like it would for SQL.

Where Does Catalog Come Into PySpark?

This depends on how you access the data.

Case 1: Reading a File Directly

```
df = spark.read.csv("/data/employees.csv")
```

Does Spark need the catalog? Not much — because you already told Spark exactly where the file is.

```
CSV File
  ↓
Infer Schema
  ↓
Logical Plan
```

Very little catalog lookup is needed.

Case 2: Reading a Managed Table

```
df = spark.table("employees")
```

Now Spark asks the Catalog: Where is employees? What format? Schema? Partition? Statistics? — Exactly like SQL.

Case 3

```
spark.sql("SELECT * FROM employees")
```

This is literally SQL. Catalog is heavily used.

What if I Join DataFrames?

```
emp.join(dept, "dept_id")
```

Catalyst still asks: Number of rows? Statistics? Table size? Join keys? Data types?

Then it decides Broadcast Join, or Sort Merge Join, or Shuffle Hash Join — exactly the same optimization process.

Does PySpark Create Logical Plans?

Yes. Every DataFrame transformation adds another node.

```
df = (
  spark.read.parquet("emp")
    .filter(col("salary") > 50000)
    .groupBy("department")
    .count()
)
```

Logical plan:

```
Aggregate
  |
Filter
  |
Read Parquet
```

Catalyst then optimizes this tree.

Why Doesn't RDD Use Catalyst?

RDD API is very low level.

```
rdd = sc.textFile("file.txt")
```

```
rdd = rdd.map(...)
```

```
rdd = rdd.filter(...)
```

Spark doesn't know what columns exist, whether it is SQL, join keys, schema, or data types. RDDs are just arbitrary objects.

RDD

↓

("John",45)

↓

Could be anything

Catalyst cannot reason about the contents. So Spark simply executes the transformations as written (while still using the DAG scheduler for execution).

SQL vs PySpark DataFrame vs RDD

Feature	SQL	PySpark DataFrame	RDD
SQL Parser	✓ Yes	✗ No	✗ No
Logical Plan	✓ Yes	✓ Yes	✗ No
Analyzer	✓ Yes	✓ Yes	✗ No
Catalog	✓ Yes	✓ Yes (when metadata is needed)	✗ No
Catalyst Optimizer	✓ Yes	✓ Yes	✗ No
Cost-Based Optimizer	✓ Yes	✓ Yes	✗ No
Physical Plan	✓ Yes	✓ Yes	✗ No
Tungsten	✓ Yes	✓ Yes	✗ No
Schema-aware	✓ Yes	✓ Yes	✗ No

Interview Question

Interviewer: Does Catalyst Optimizer work only for SQL?

Answer: "No. Catalyst Optimizer is used by both Spark SQL and the DataFrame/Dataset APIs. SQL queries are first parsed into a logical plan by the SQL parser; while DataFrame operations build the logical plan directly through the API. After that point, both follow the same pipeline: Analyzer; Catalog (when metadata is needed), Catalyst Optimizer; Physical Planner; Cost-Based Optimizer; Tungsten, and finally execution on the executors. The only major Spark API that does not use Catalyst is the RDD API, because RDDs are schema-less and Spark cannot apply relational query optimizations."